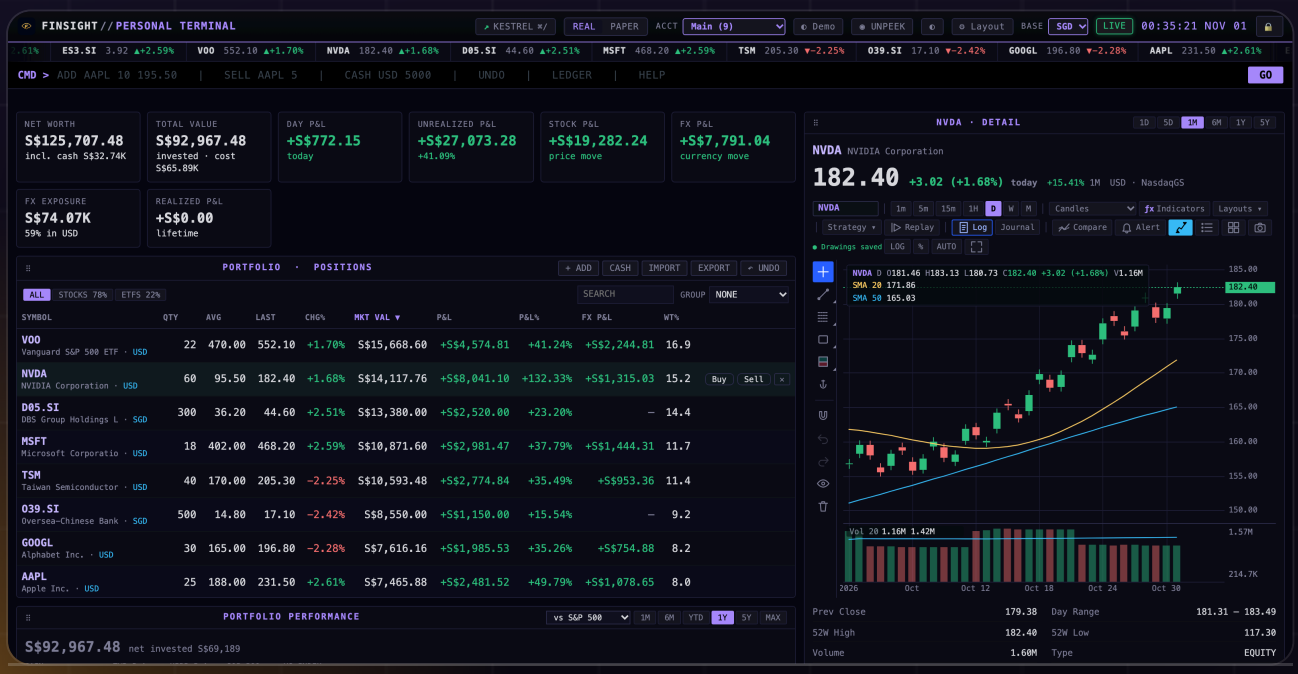
FINSIGHT // TERMINAL

MASTER PRODUCT & ENGINEERING DOCUMENT

FinSight Terminal & Kestrel Agent

Product · Architecture · Agent · Security · Quality

Product, requirements, architecture, agent design, security, concurrency, quality and operations in one place.



VERSION	DATE	OWNER	STATUS
1.0	1 November 2026	tanueihorng	Living document · reflects main on 1 November 2026

Contents

Document control

THE PRODUCT

1 Executive summary

2 Problem and motivation

3 Product overview

4 Stakeholders and personas

PART A · PRODUCT AND DESIGN FOUNDATIONS

5 Product requirements and scope

6 User stories

7 User flows

8 Functional specification

9 Non-functional requirements

10 System architecture

11 Technology stack

12 Frontend engineering and UX

PART B · APPLICATION LOGIC AND DATA

13 Backend engineering

14 Database and data architecture

15 APIs and system integration

16 Identity and access management

PART C · AI ENGINEERING AND AGENTIC SYSTEMS

17 Kestrel agent design

18 Investment council design

19 Paper trading design

PART D · SECURITY, CORRECTNESS AND SYSTEM BEHAVIOUR

20 Application security

21 Concurrency and parallelism

22 Performance and scalability

23 Reliability and fault tolerance

24 Testing and quality assurance

25 UAT plan for computer-use testing

PART E · DELIVERY, INFRASTRUCTURE AND OPERATIONS

26 SDLC and governance

27 Infrastructure and environments

28 CI/CD and release engineering

29 Observability and monitoring

30 Production operations and support

PART F · BUSINESS, GOVERNANCE AND SUSTAINABILITY

31 Cost engineering and FinOps

32 Privacy, legal and compliance

33 Documentation and knowledge management

34 Maintainability and future evolution

35 Risks, assumptions and constraints

APPENDICES

Appendix A — Glossary

Appendix B — Terminal commands

Appendix C — Kestrel commands

Appendix D — Traceability matrix

Appendix E — References

Document control

Field	Value
Document	FinSight Terminal — Master Product & Engineering Document
Version	1.0 (1 November 2026)
Owner	tanueihorng (product owner, sole user, final decision-maker)
Code baseline	`main` on 1 November 2026: the FEAT-012 merge (`aa373b7`) plus the MCP read-only fix
Audience	New contributors, coding agents, reviewers, demo audiences and UAT testers (including Codex computer use)
Companion files	`docs/guide/finsight-guide.html` (interactive demo guide), `agent/docs/SPEC.md`, `docs/architecture/SYSTEM.md`, `docs/BACKLOG.md`

> **Naming**
>
> **Kestrel** is FinSight's built-in AI research agent (sometimes misheard as "Cashier" or "Cursor" in voice notes). **Council** is Kestrel's multi-specialist review mode. **Paper trading** is the Alpaca-backed fake-money book.

> **Status words**
>
> This document distinguishes **implemented** (code exists), **verified** (tests/evidence recorded), **merged** (on `main`) and **deployed** (running in the owner's personal install). Figures shown in examples come from the synthetic demo portfolio, not from real holdings.

How to read this document

The first part explains **the product**: the problem FinSight solves, what the terminal and the Kestrel agent do, and who they are for. The rest of the document is organised by the **24 software-engineering domains** a complete application has to address, from requirements to long-term ownership. Each domain has its own chapter, written for FinSight specifically and checked against `main` on 1 November 2026.

- **Part A — Product and design foundations:** requirements and scope, user stories and flows, the functional and non-functional specifications, architecture, technology stack, frontend and UX.
- **Part B — Application logic and data:** backend, data architecture, APIs and integrations, identity and access.
- **Part C — AI engineering and agentic systems:** the Kestrel agent, the investment council and agentic paper trading.
- **Part D — Security, correctness and system behaviour:** security, concurrency, performance, reliability, testing and the UAT plan.
- **Part E — Delivery and operations:** SDLC, infrastructure, CI/CD, observability, operations and support.
- **Part F — Business and sustainability:** cost, privacy and compliance, documentation, analytics, maintainability, risks.

No.	Engineering domain	Where this document covers it
01	Product requirements and scope	§5 Product requirements and scope; §6 User stories; §7 User flows; §8 Functional specification; §9 Non-functional requirements
02	System architecture	§10 System architecture
03	Technology stack selection	§11 Technology stack
04	Frontend engineering and UX	§12 Frontend engineering and UX
05	Backend engineering	§13 Backend engineering
06	Database and data architecture	§14 Database and data architecture
07	APIs and system integration	§15 APIs and system integration; §19 Paper trading design
08	Identity and access management	§16 Identity and access management
09	Application security	§20 Application security
10	Concurrency and parallelism	§21 Concurrency and parallelism
11	Performance and scalability	§22 Performance and scalability
12	Reliability and fault tolerance	§23 Reliability and fault tolerance
13	Testing and quality assurance	§24 Testing and quality assurance; §25 UAT plan for computer-use testing
14	SDLC and governance	§26 SDLC and governance
15	Infrastructure and cloud	§27 Infrastructure and environments
16	CI/CD and release engineering	§28 CI/CD and release engineering
17	Observability and monitoring	§29 Observability and monitoring
18	Production operations and support	§30 Production operations and support
19	Cost engineering and FinOps	§31 Cost engineering and FinOps
20	Privacy, legal and compliance	§32 Privacy, legal and compliance

No.	Engineering domain	Where this document covers it
21	Documentation and knowledge	§33 Documentation and knowledge management
22	Product analytics and experiments	§32 Privacy, legal and compliance
23	Maintainability and evolution	§34 Maintainability and future evolution
24	AI engineering and agentic systems	§17 Kestrel agent design; §18 Investment council design; §19 Paper trading design



Figure 1 — How the engineering domains sit on FinSight's layers. Security, concurrency, testing and operations cut across every layer.

FINSIGHT

The product

What FinSight and Kestrel are, the problem they solve and who they are for

1. Executive summary

FinSight is a free, Bloomberg-style portfolio terminal that runs entirely on the owner's computer. It tracks a multi-currency, multi-account portfolio with live public market data, and it answers a question most tools hide: **how much of my return came from the stock, and how much from the currency?**

On top of the terminal sits ↗ **Kestrel**, a local AI research agent that can see what the user sees — holdings, cash, P&L, events, strategies and the chart in view — and answers with numbers produced by deterministic code, with sources and timestamps. For important decisions, Kestrel can convene an **investment council**: three specialists and a chair who assess, challenge and synthesise. A **paper-trading** mode, backed by an Alpaca paper account, lets the user and Kestrel practise with fake money under explicit approval and limits.

0	0	≈38	3 + 1	2	762
runtime deps (server)	API keys for market data	agent tools	council specialists + chair	books: REAL · PAPER	automated tests

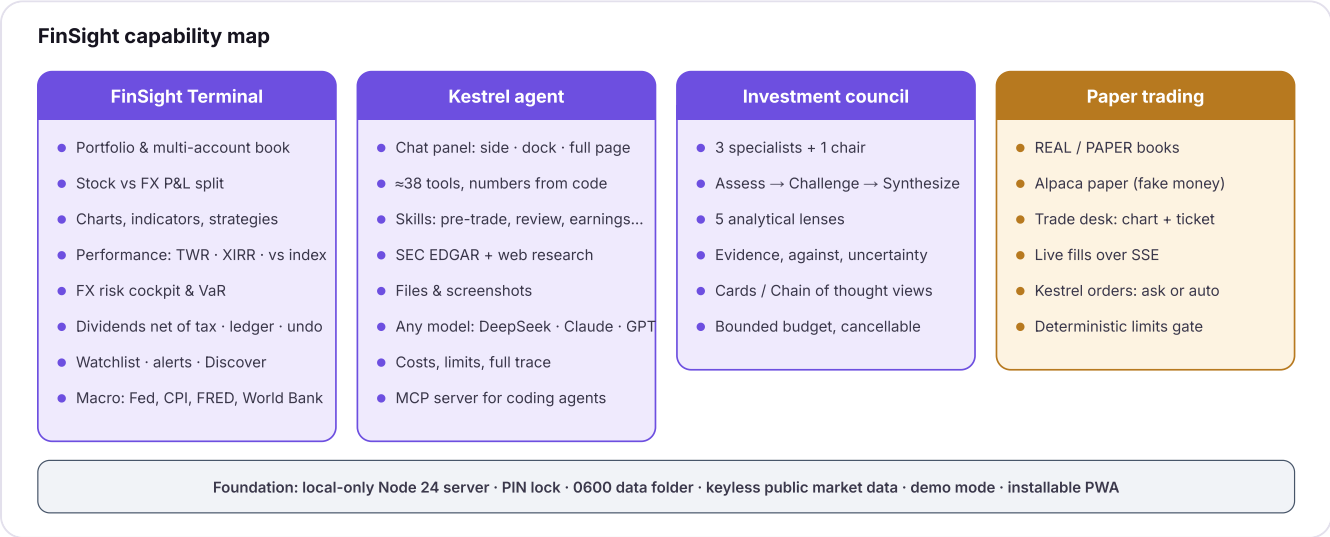
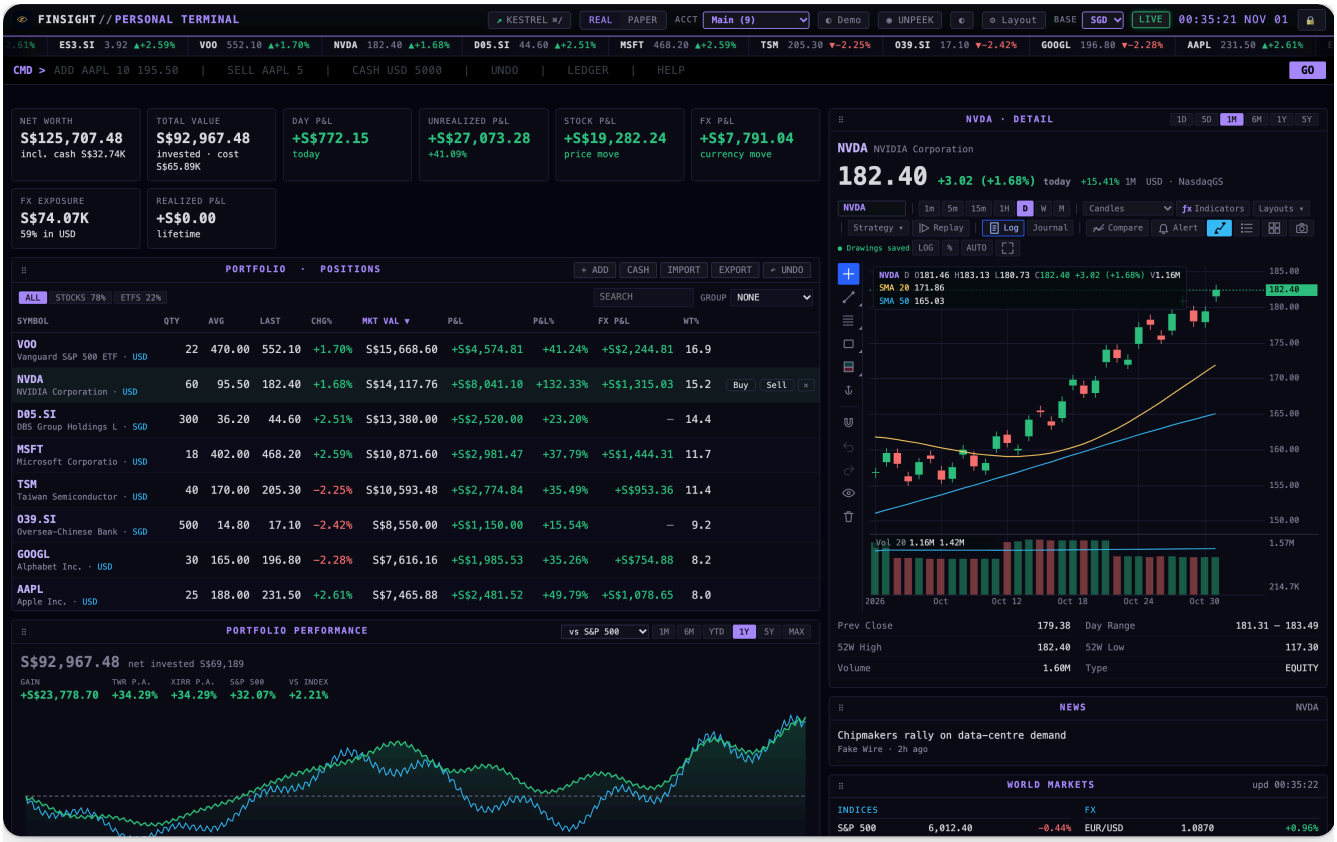
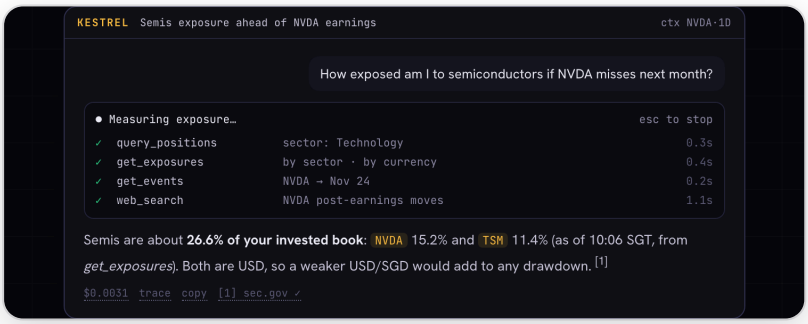


Figure 2 — Capability map



LIVE CAPTURE Screen 1 — The FinSight Terminal running the demo portfolio: summary cards with the stock/FX P&L split, live positions, the NVDA chart and performance vs the S&P 500



GUIDE RECONSTRUCTION Screen 2 — Kestrel answering a portfolio question: each tool call is shown, the figures come from tools, and the answer carries an as-of time, sources and its cost

2. Problem and motivation

2.1 The problem

A retail investor who **funds in one currency and invests in another** (for example converting SGD to buy US stocks) faces four recurring problems:

1. **Hidden currency effect.** Broker screens report gains in one currency. The investor cannot see how much of a gain or loss was the share price and how much was USD/SGD moving.
2. **Fragmented view.** Holdings sit across brokers and accounts (IBKR, moomoo, Tiger, SGX). No single local view rolls them up in the base currency with true time-weighted and money-weighted returns.
3. **Expensive or privacy-hostile tools.** Professional terminals are costly; free portfolio sites require uploading holdings to a third party.
4. **Untrustworthy AI help.** General chatbots invent numbers, do arithmetic badly, forget the user's rules and cannot see the actual book.

2.2 Motivation and vision

FinSight's vision is "**a trading floor for one, running entirely on this machine**": the depth of a professional terminal, the privacy of a local app, and an AI analyst that is grounded in the user's real data and rules and that shows its work.

2.3 Goals

ID	Goal	Measure of success
G1	See the whole book in one base currency	All accounts roll up; every holding shows stock P&L and FX P&L
G2	Make currency risk explicit	Blended entry rate, FX P&L, what-if shock and 1-week VaR available
G3	Stay private and free	No account, no telemetry, keyless data, data folder outside the repo
G4	Ground AI answers in facts	Every figure comes from a tool; answers cite asOf/sources; cost visible
G5	Support better decisions, not automate them	Council gives structured views; the user decides every real trade
G6	Allow safe practice	Paper book with approval/limits; no live-trading path in code

2.4 Non-goals

- Placing real-money orders with any broker. FinSight records the real book; it never sends real orders.
- Multi-user or hosted SaaS operation. FinSight is single-user and local.
- Licensed, real-time market data. Public endpoints are delayed and best-effort.

- Investment advice. Outputs are research aids; the user is the decision-maker.

3. Product overview

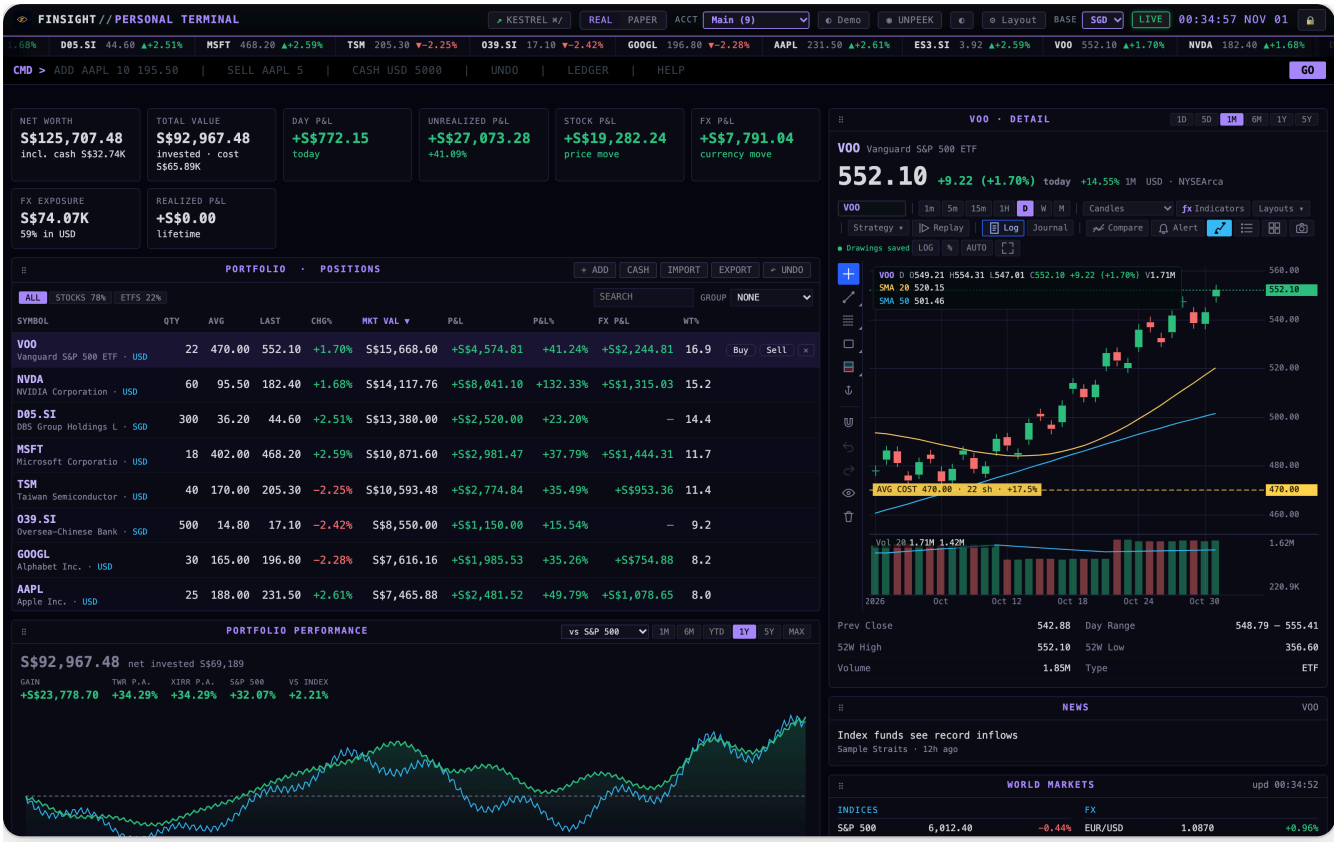
3.1 FinSight Terminal

A local web application (served by `node server.ts` on port 8000) with a dark, themeable terminal UI. Key areas:

Area	What the user gets
Top bar	↗ KESTREL (⌘/), REAL/PAPER switch, account picker (All Accounts / + New), Profile, UNPEEK privacy mode, light/dark, ⚙ Layout, base currency, live status, lock
Command bar	Bloomberg-style commands: `ADD`, `SELL`, `DEL`, `CASH`, `UNDO`, `LEDGER`, `Q`, `WATCH`, `ALERT`, `NEWS`, `WHT`, `MKT`, `HELP`
Summary cards	Net worth, total value, day P&L, unrealized P&L, **stock P&L**, **FX P&L**, FX exposure, realized P&L
Positions	Live table with weight and FX P&L, filters, grouping, Buy/Sell trade sheet, import/export, undo
Security detail	Canvas chart: candle types, indicators (SMA/EMA/Bollinger/RSI/MACD/volume), strategies, replay, journal, compare, drawing tools, chart alerts
Analytics	Performance (TWR, XIRR, vs S&P 500 / MSCI World / All-World / Nasdaq / STI in base currency), net-worth breakdown, allocation donut, sector heatmap
FX Risk	USD exposure, blended entry, spot drift, FX P&L, per-buy FX history, what-if slider, 1σ/95% weekly VaR, target exposure, convert now vs later
Income & history	Dividends net of withholding tax (yield on cost, next ex-date), recent activity, ledger with realized gains by year
Watch & alert	Watchlist, price/trend/RSI/MA alerts with desktop notification, toast and beep
Discover	Movers among held/watched symbols, US market movers (Yahoo screens), US IPO calendar (Nasdaq), upcoming earnings
Macro	World markets, econ calendar (next Fed decision and dot plot, CPI/PCE/jobs/GDP), FRED series, World Bank indicators, inflation gauges
Personalisation	Movable panels, accent palettes (Violet/Amber/Teal/Phosphor), Mono/Sans/Serif fonts, light mode

The terminal, filled with the demo portfolio

The screens below come from the running app with the synthetic demo book (9 holdings across USD and SGD, fake market data). No real holdings are shown.



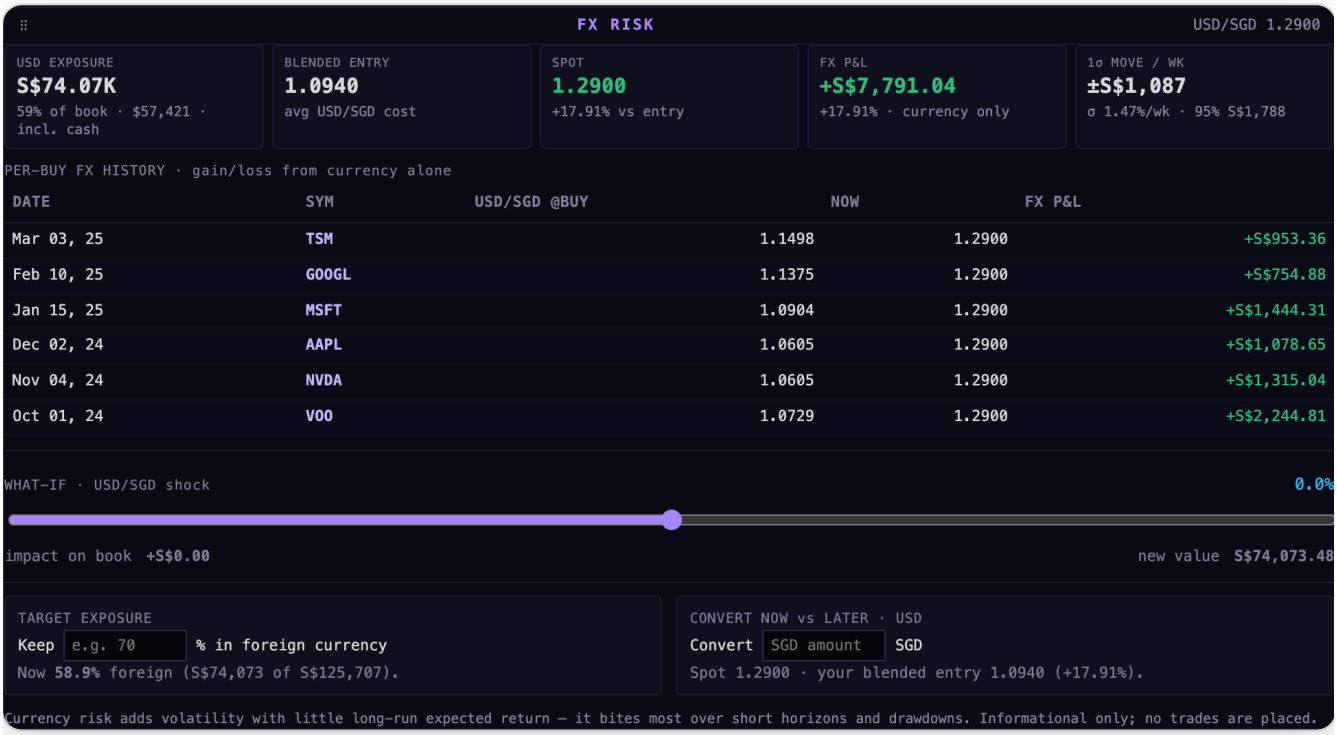
LIVE CAPTURE Screen 3 — Terminal home: top bar (Kestrel, REAL/PAPER, account, UNPEEK, layout, base currency), ticker, command bar, summary cards and positions



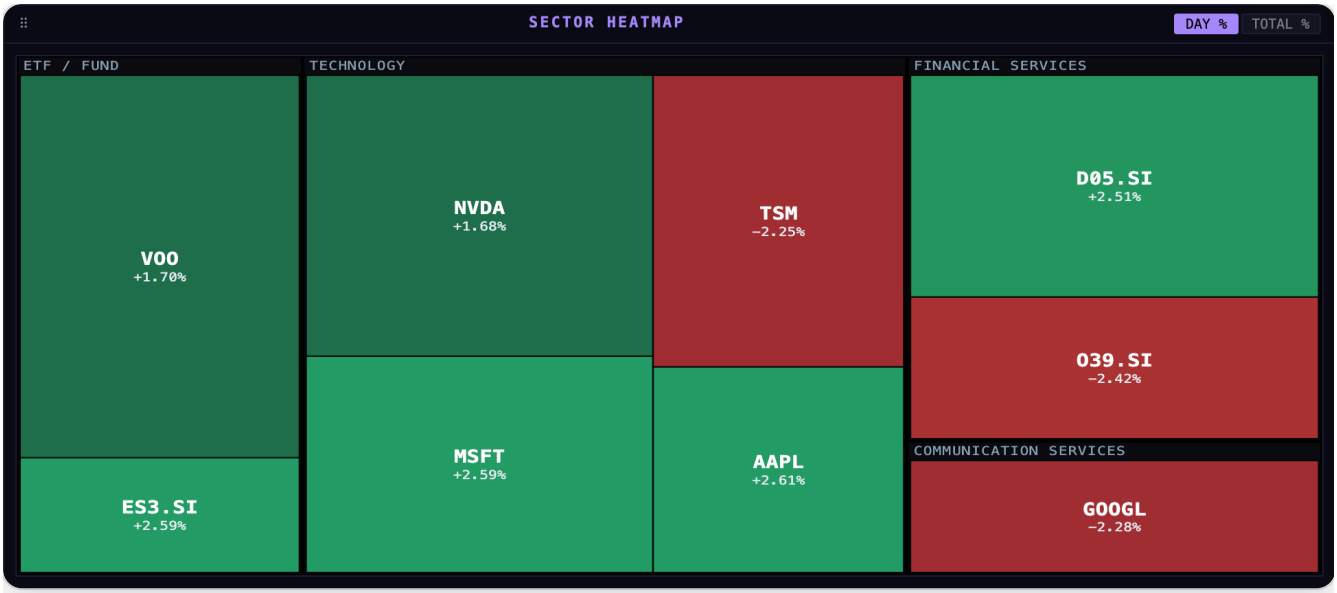
LIVE CAPTURE Screen 4 — Positions: quantity, average cost, last price, market value in SGD, P&L, P&L %, FX P&L and weight per holding



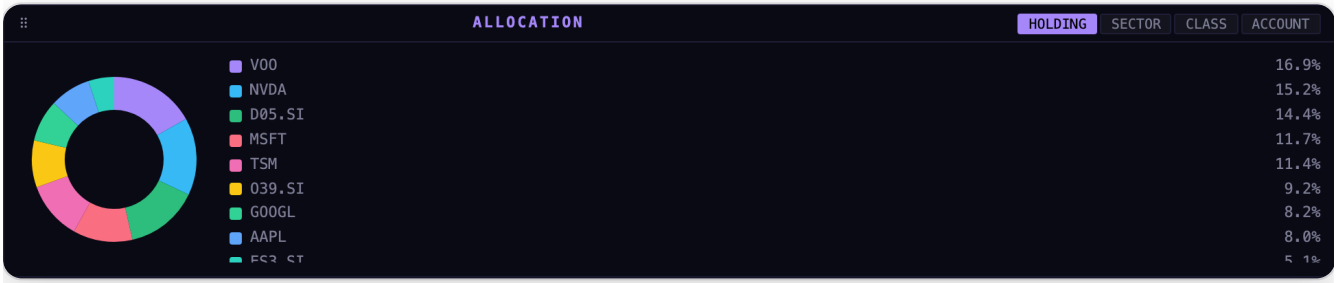
LIVE CAPTURE Screen 5 — Performance: time-weighted and money-weighted return against the S&P 500, in the base currency



LIVE CAPTURE Screen 6 — FX Risk: USD exposure, blended entry rate, spot drift, FX P&L, per-buy history, a what-if shock slider and conversion planning



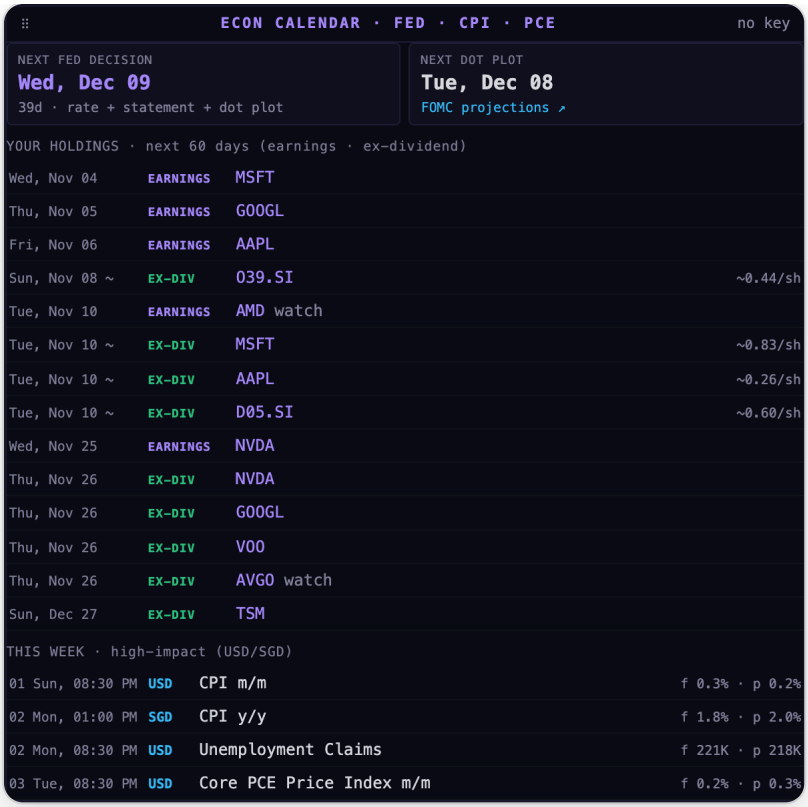
LIVE CAPTURE Screen 7 — Sector heatmap: tile size is position weight, colour is the day's move



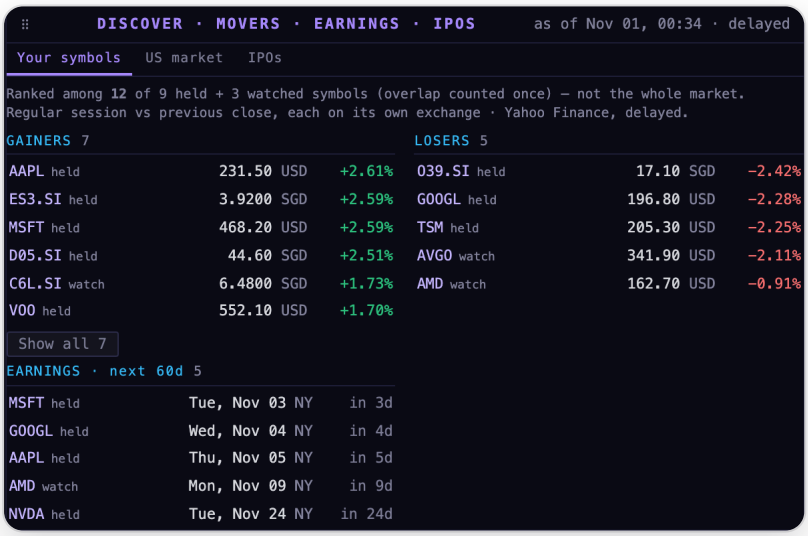
LIVE CAPTURE Screen 8 — Allocation by holding (switchable to sector, asset class or account)



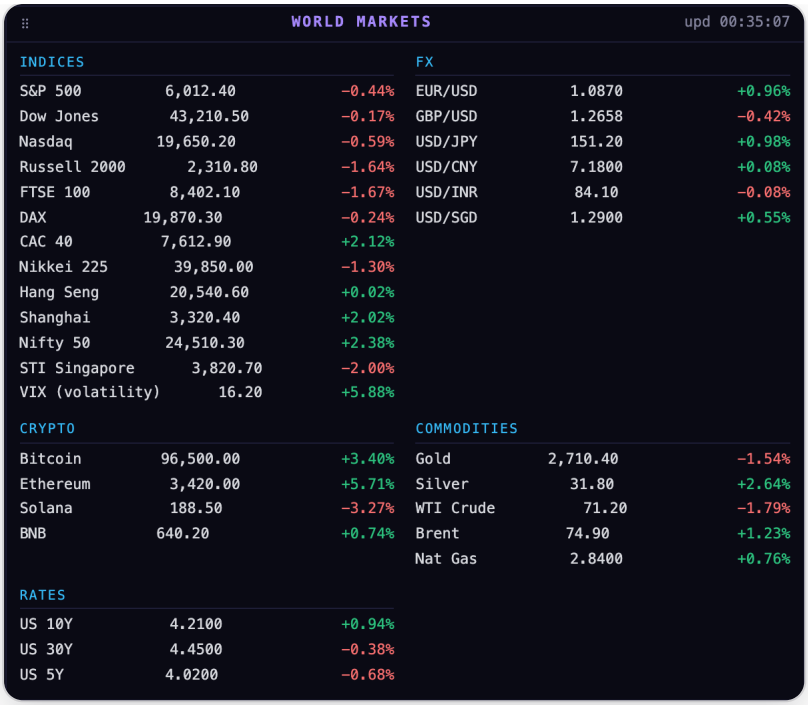
LIVE CAPTURE Screen 9 — Dividends: forward income net of withholding tax, yield on cost and the next ex-date



LIVE CAPTURE Screen 10 — Econ calendar: next Fed decision and dot plot, holdings' earnings and ex-dividend dates, this week's high-impact releases



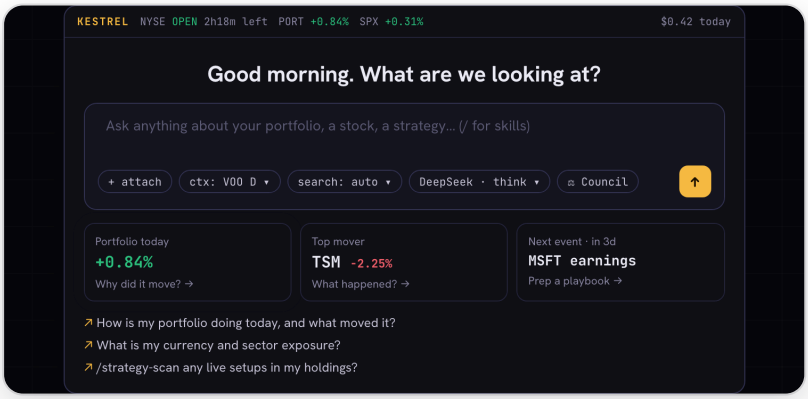
LIVE CAPTURE Screen 11 — Discover: movers among held and watched symbols, and upcoming earnings



LIVE CAPTURE Screen 12 — World markets: indices, FX, crypto, commodities and rates

3.2 ↗ Kestrel — AI research agent

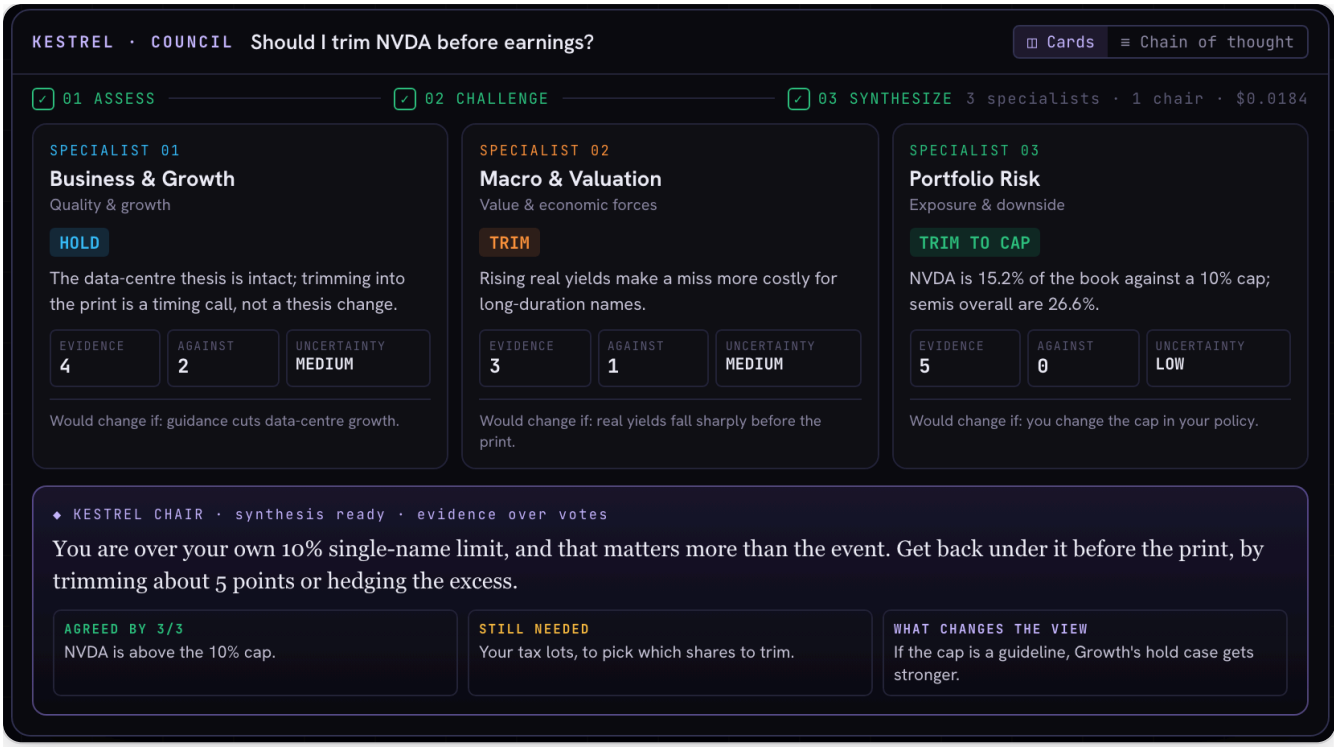
A supervised local service (port 8100) behind FinSight's PIN. It chats in a panel that can be a side drawer, a docked terminal panel, a full page with a workspace column, or a collapsed capsule. It uses ≈38 tools for portfolio facts, strategies, SEC EDGAR, web research, files, records and (in paper mode) paper orders. Skills (`/pre-trade-check` , `/position-review` , `/earnings-review` , `/why-did-it-move` , `/strategy-scan`) run repeatable playbooks. It works with DeepSeek (default), Claude, OpenAI-compatible models or the Claude Code CLI, shows cost per answer and keeps a full trace.



GUIDE RECONSTRUCTION Screen 13 — Kestrel start screen: the ask box with attach, context, search mode, model and ⚖ Council, plus briefing tiles built from the book

3.3 Investment council

An explicit mode (⚖ Council or `/committee`) in which **Business & Growth**, **Macro & Valuation** and **Portfolio Risk** specialists assess a decision independently, run a challenge round, and a **Kestrel Chair** synthesises what they agree on, what is still needed and what would change the view. Five lenses (Balanced, Growth, Speculative/Catalyst, Hedge Fund, Macro) tune emphasis without changing rules or data.



GUIDE RECONSTRUCTION Screen 14 — Council result, Cards view: each specialist's verdict, evidence and uncertainty, with the chair's synthesis below

3.4 Paper trading

A second book next to the real one. The user connects an Alpaca **paper** key pair; FinSight mirrors Alpaca's fills into a paper portfolio so every existing view works. A trade desk (chart, docked ticket, positions/orders/history/Kestrel/settings tabs) places market and limit orders. Kestrel can place paper orders too: **Ask me first** (pending ticket to approve) or **Autonomous**, inside optional deterministic limits.



LIVE CAPTURE Screen 15 — Paper trade desk after real fake-broker orders: account strip, AAPL chart with buy/sell fill markers and an open limit line, the docked ticket and four paper positions

3.5 Demo mode

`npm run demo` starts an isolated FinSight and Kestrel on port 8003 with a synthetic portfolio (PIN 2468) in `~/FinSightDemo`. It never touches real data and is the recommended environment for demos and UAT.

4. Stakeholders and personas

Persona	Description	Primary needs
Owner-investor (primary)	Technically capable individual investor in Singapore, funds in SGD, holds US/SG/HK equities and ETFs across brokers	One private view; FX clarity; grounded research; rules enforced; safe practice
Demo audience	Friends, peers or reviewers watching a walkthrough	Understand value quickly without seeing real holdings (demo mode, UNPEEK)
Contributor / coding agent	Developer or AI coding agent (Claude Code, Codex, Cursor) changing the code	Clear specs, boundaries, quality gates, docs-as-code process (AGENTS.md)
UAT tester	Human or Codex computer-use agent executing acceptance scripts	Deterministic environment (demo + fake market), precise steps and expected results (§25)
External MCP client	Claude Code / Codex / Cursor using FinSight tools over MCP	Kestrel's read-only tools (paper order tools excluded) and the same citation rules

Single-user by design

There is exactly one user class with full authority. There is no multi-tenancy, no roles and no remote access. All security controls assume an attacker is another process, web page or document on the same machine — not a second legitimate user.

PART A

Product and design foundations

Domains 01–04: requirements, architecture, technology stack, frontend and UX

5. Product requirements and scope

`Domain 01 Product requirements and scope`

This chapter fixes what FinSight is for and where its edges are. The user stories, flows and specifications in the next chapters elaborate it.

5.1 Problem statement and business goals

A Singapore-based investor who funds in SGD and buys US, SG and HK securities cannot see how much of a return came from the stock and how much from the currency. Holdings are spread across brokers, professional tools are expensive, and general chatbots invent numbers. FinSight's goals G1–G6 (§2) turn this into measurable outcomes: one base-currency view, explicit FX risk, privacy at no cost, AI answers grounded in tools, decision support rather than automation, and safe practice with fake money.

5.2 Scope

In scope	Out of scope (by decision)
Multi-account, multi-currency portfolio book with lot-level cost and purchase FX	Real-money order routing to any broker
Analytics: stock vs FX P&L, TWR/XIRR vs index, FX risk, dividends after tax	Multi-user, hosted or SaaS operation
Market data, charting, strategies, alerts, discovery and macro panels from free sources	Licensed real-time data feeds
Kestrel research agent with tools, skills, files, limits and MCP	Investment advice; the user decides every trade
Investment council for structured decision reviews	Shorts, options and margin in the book
Paper trading on Alpaca paper with approval and limits	Scheduled or autonomous agent runs (planned as FEAT-013)

5.3 Prioritisation and roadmap

Work is prioritised in `docs/BACKLOG.md`. An idea enters the icebox as IDEA-xxx; nothing is built from an idea. When selected, it becomes a feature spec FEAT-xxx with intent, decisions, scope, out-of-scope and

acceptance criteria, and is scheduled with a task id (for example PAPER-001). The MVP was the local terminal with the stock/FX split; Kestrel, the council and paper trading followed as separately specified increments. Current status and the roadmap are in §34.

5.4 Business rules and constraints

- **No model arithmetic.** Every total, weight, size and return is computed by deterministic code; the model only explains (`AGENTS.md`).
- **The user decides.** Kestrel never places real orders and never edits records without confirmation; paper orders follow the approval mode.
- **Base currency is the unit of truth.** Unrealized P&L = Stock P&L + FX P&L in the base currency, using each lot's purchase-time FX.
- **Lots are kept.** Buys add lots; sells realise P&L against lots; transfers between accounts keep lots, cost and FX.
- **Paper and real never mix.** The PAPER book mirrors Alpaca fills only; manual edits are refused in PAPER mode.
- **No personal data in the repository.** Holdings, keys and broker exports live only in the data folder.

5.5 Edge cases the requirements cover

Edge case	Required behaviour
Back-dated trade	Fetch the historical FX rate for the trade date (FR-PF-7)
Duplicate broker import	IBKR fills previewed with duplicate detection before saving (FR-PF-5)
Sell more than held / sell an unknown symbol	Refused with a message; nothing written
Market data source down or rate-limited	Last good values served and flagged stale
Ambiguous company in a council question	Clarification or explicit gap, never a guess from the chart in view (FR-C-6)
Kestrel order breaching a limit	Refused before reaching the broker, with the reason (FR-P-6)
Live Alpaca key entered	Refused before saving (FR-P-1)

5.6 Acceptance criteria and definition of done

Every story in §6 carries acceptance criteria, and every functional requirement in §8 traces to tests (Appendix D). A change is done when its feature spec's acceptance criteria are met, `npm run verify` and `npm run agent:test` pass, the affected documentation is updated in the same change, and `CHANGELOG.md` records it with its commit reference.

5.7 Success metrics

FinSight has no telemetry by design (see §32), so success is measured by observable outcomes rather than usage analytics: every holding shows a stock/FX split; Kestrel answers cite an as-of time and

sources and show cost; daily AI spend stays under the cap; council runs complete or fail with an explicit gap; and the UAT catalogue (§25) passes.

5.8 Stakeholder alignment and change management

The owner is the single decision-maker. Changes to agreed behaviour are recorded as dated decisions inside the feature spec; changes that contradict `agent/docs/SPEC.md` need the owner's approval before code. Coding agents follow `AGENTS.md`, so human and AI contributors work from the same rules.

6. User stories

`Domain 01 Product requirements and scope`

Stories follow "As a … I want … so that …" with acceptance criteria (AC). IDs map to functional requirements (§8) and UAT cases (§25).

Epic A — Access and setup

ID	Story	Acceptance criteria
US-A1	As the owner, I want to lock FinSight behind a PIN so that nobody using my computer can read my portfolio.	PIN required for all `/api/*` data; wrong PIN shows "Wrong PIN"; repeated failures are throttled; 🔒 locks immediately
US-A2	As the owner, I want to choose REAL or PAPER before unlocking so that I land in the right book.	Lock screen offers both; choice persists across tabs and reloads; PAPER is visibly marked
US-A3	As a new user, I want a guided first-run so that I can start in minutes.	Operator card asks name and base currency; offers CSV import, manual add or empty start; reopen from Profile
US-A4	As a presenter, I want a demo mode so that I can show FinSight without exposing my data.	`npm run demo` opens port 8003 with synthetic data and PIN 2468; real data folder untouched

Epic B — Portfolio and analytics

ID	Story	Acceptance criteria
US-B1	As the owner, I want every holding valued in my base currency so that I see one total.	Summary cards and positions use BASE; changing BASE re-prices everything
US-B2	As the owner, I want gains split into stock and FX parts so that I know what drove returns.	Unrealized = Stock P&L + FX P&L; per-position FX P&L column; lots store purchase FX
US-B3	As the owner, I want to record buys and sells quickly so that the book stays accurate.	Command bar and trade sheet record trades; sheet previews position after and new average; UNDO reverses
US-B4	As the owner, I want to import broker statements so that I don't retype holdings.	Plain CSV, IBKR, moomoo, Tiger supported; IBKR fills previewed with duplicate detection
US-B5	As the owner, I want multiple accounts and transfers so that the book matches reality.	Create accounts; All Accounts view; transfers keep lots, cost and FX; undoable
US-B6	As the owner, I want true performance vs an index so that I know if I'm adding value.	TWR, XIRR, gain, vs index in base currency for 1M… MAX; benchmark selectable
US-B7	As the owner, I want to see FX risk so that I can plan conversions.	FX Risk shows exposure, blended entry, spot drift, VaR, what-if, target, convert-now
US-B8	As the owner, I want dividend income after tax so that I can plan cash flow.	Forward income net of WHT; yield on cost; next ex-date; `WHT` command changes rates

Epic C — Markets, discovery and alerts

ID	Story	Acceptance criteria
US-C1	As the owner, I want a full chart for any symbol so that I can analyse price action.	Click a row or `Q SYM`; timeframes, chart types, indicators, strategies, drawings saved
US-C2	As the owner, I want price alerts so that I don't have to watch the screen.	`ALERT AAPL > 320` or UI; fires notification, toast and beep
US-C3	As the owner, I want to see what's moving and what's coming so that I can prepare.	Discover tabs (Your symbols, US market, IPOs) with source, as-of and stale flags; rows open detail
US-C4	As the owner, I want the macro calendar so that I know when rates news lands.	Next Fed decision/dot plot always shown; this week's high-impact releases; holdings' earnings/ex-dates
US-C5	As a presenter, I want to hide money figures so that I can share my screen.	UNPEEK blanks every money figure including Kestrel/council views

Epic D — Kestrel agent

ID	Story	Acceptance criteria
US-D1	As the owner, I want to ask questions about my portfolio in plain language so that research is fast.	⌘/ opens Kestrel; answers stream; figures come from tools with asOf; sources cited
US-D2	As the owner, I want to see what Kestrel did so that I can trust or challenge it.	Tool calls shown live; per-answer cost; trace of every model/tool/data call
US-D3	As the owner, I want repeatable playbooks so that important checks are never skipped.	`/` menu lists skills; `/pre-trade-check` sizes from real cash and checks concentration and events
US-D4	As the owner, I want to choose the model and thinking effort so that I balance cost and depth.	Model picker and effort menu; `/model`; cost per answer and per day visible
US-D5	As the owner, I want spending limits so that a runaway loop can't cost much.	Per-turn and per-day caps editable (`/limits`); limits enforced before billable calls
US-D6	As the owner, I want to give Kestrel files and screenshots so that it can analyse them.	Attach/paste/drop PDFs, DOCX, CSV, images; content treated as untrusted data
US-D7	As the owner, I want Kestrel to remember my rules only with my consent so that records stay mine.	Stated facts become `propose_record_update` proposals; nothing changes until confirmed
US-D8	As a developer, I want FinSight tools in my coding agent so that I can query data from my IDE.	`claude mcp add finsight -- node agent/src/cli.ts mcp` exposes the registry over stdio

Epic E — Council

ID	Story	Acceptance criteria
US-E1	As the owner, I want several expert views on a decision so that I avoid one-sided reasoning.	Council runs 3 specialists + chair through Assess → Challenge → Synthesize

ID	Story	Acceptance criteria
US-E2	As the owner, I want to set an analytical lens so that the review fits my question.	Lens picker (5 lenses) shown when Council is on; lens recorded with the turn
US-E3	As the owner, I want to see evidence and uncertainty so that I can judge each view.	Cards show verdict, evidence/against counts, uncertainty, would-change-if; Chain of thought view
US-E4	As the owner, I want the council to admit gaps so that I'm not misled.	Ambiguous company or missing data → clarification/gap, partial status; unverified figures flagged

Epic F — Paper trading

ID	Story	Acceptance criteria
US-F1	As the owner, I want a paper book so that I can practise without risking money.	PAPER mode with Alpaca paper keys; real book untouched; manual book edits refused
US-F2	As the owner, I want a professional trade desk so that placing orders is quick and clear.	Account strip, chart linked to ticket, BUY/SELL, Market/Limit, shares or US$, quick sizes, TIF, estimate
US-F3	As the owner, I want fills to appear live so that I don't refresh.	trade_updates → book resync → SSE; positions, ledger, P&L, chart markers update
US-F4	As the owner, I want Kestrel to propose or place paper orders under my control so that I can test agentic trading safely.	Ask mode creates pending tickets (Approve/Reject); Auto places directly; limits refuse with reason; only the PIN session can change settings

7. User flows

Domain 01 Product requirements and scope

The five flows below cover every primary journey. §25 turns each into executable UAT scripts.

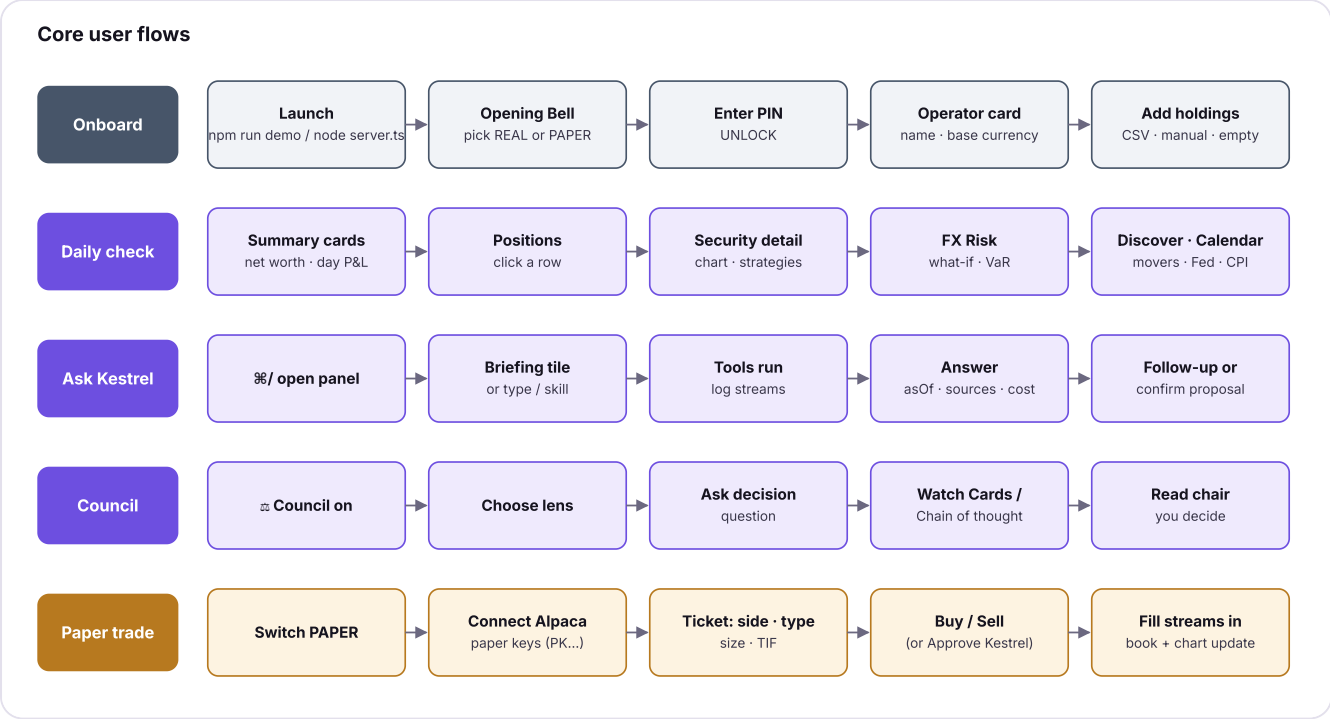


Figure 3 — Core user flows

Flow narratives

Flow	Entry point	Happy path	Key alternate paths
Onboard	Launcher or `node server.ts`	Opening Bell → choose book → PIN → operator card → import → terminal	Wrong PIN (throttled) · start empty · reopen via Profile · forgot PIN (delete auth.json)
Daily check	Terminal home	Summary cards → positions → click row → chart → FX Risk → Discover → calendar	Stale quotes flagged · UNPEEK for sharing · switch account or base
Ask Kestrel	⌘/ or ↗ KESTREL	Briefing tile or question → tool log → answer with asOf, sources, cost	Agent offline hint · stop with Esc · limits reached · proposal confirm/dismiss
Council	⚖ Council pill or `/committee`	Lens → decision question → cards/chain stream → chair synthesis	Ambiguous subject → clarification · budget refusal · cancel aborts tree
Paper trade	PAPER (lock screen or top bar)	Connect keys → ticket → submit → fill streams → book updates	Live keys refused · limit refusal · Kestrel ticket approve/reject · cancel open order

8. Functional specification

`Domain 01 Product requirements and scope`

Requirements use MUST for mandatory behaviour. Each FR traces to stories (§6) and tests (Appendix D).

Access, session and modes

ID	Requirement
FR-ACC-1	The server MUST gate every data API behind a PIN session; the UI shell may load without it.
FR-ACC-2	PINs MUST be stored scrypt-hashed in `auth.json`; failed attempts MUST be throttled.
FR-ACC-3	Session cookies MUST be `HttpOnly; SameSite=Strict`; idle lock after `LOCK_IDLE_MIN` minutes when configured.
FR-ACC-4	The book mode (REAL/PAPER) MUST be stored in `book.json` and apply to every tab and to Kestrel.
FR-ACC-5	First run MUST offer the operator card (name, base currency, CSV/manual/empty).

Portfolio, trades and imports

ID	Requirement
FR-PF-1	Positions MUST be lot-based (`q, px, fxUsd, t`), preserving purchase-time FX for return decomposition.
FR-PF-2	Unrealized P&L MUST equal Stock P&L + FX P&L in the base currency.
FR-PF-3	Buy/sell/edit/delete/cash/import/transfer MUST be undoable (25-step undo stack).
FR-PF-4	The trade sheet MUST support new symbols, show position-after and new average before saving.
FR-PF-5	Imports MUST accept plain CSV and IBKR, moomoo and Tiger statements; IBKR fills MUST be previewed with duplicate detection.
FR-PF-6	Transfers between accounts MUST preserve lots, cost basis and FX; the ledger MUST report realized gains by year and export CSV.
FR-PF-7	Back-dated trades MUST fetch the historical FX rate for the trade date.

Market data, chart and analytics

ID	Requirement
FR-MD-1	Quotes, history, FX, search and news MUST come from keyless public sources (Yahoo Finance) with stale-while-revalidate caching and 429 backoff.
FR-MD-2	The chart MUST support 1m–M intervals, six chart types, indicators, strategy overlays, replay, journal, compare, drawings and alerts.
FR-AN-1	Performance MUST replay actual trades with historical prices and FX to compute TWR and XIRR, and compare against a chosen index in base currency.

ID	Requirement
FR-AN-2	FX Risk MUST show exposure, blended entry, spot drift, FX P&L, per-buy history, what-if shock, 1σ and 95% weekly VaR, target exposure and convert-now estimate.
FR-AN-3	Dividends MUST be forward-estimated net of per-market withholding (default US 30%), with yield on cost and next ex-date.
FR-DS-1	Discover MUST label its universe, baseline, session, as-of and stale/failed states; US movers via Yahoo screens; IPOs via Nasdaq with EDGAR links.
FR-MC-1	The econ calendar MUST always show the next FOMC decision and dot-plot date, plus this week's high-impact releases.
FR-AL-1	Alerts MUST fire desktop notification, toast and beep for price, trend line, RSI and moving-average rules.

Kestrel agent

ID	Requirement
FR-K-1	Kestrel MUST run as a supervised loopback service reachable only through FinSight's `/api/agent/*` proxy.
FR-K-2	All totals, weights and sizes MUST come from tools (`query_positions`, `get_exposures`, `size_position`...), never model arithmetic.
FR-K-3	Answers MUST cite `asOf` / `retrievedAt` for data and source URL + date for web facts; missing data MUST be reported.
FR-K-4	Kestrel MUST stream tool calls and text over SSE with event ids for replay; Esc/■ MUST stop a turn.
FR-K-5	Users MUST be able to switch model and thinking effort; each answer MUST show its cost; `/cost` reports spend.
FR-K-6	Limits (defaults: 25 steps, 300 s, US$0.25 per turn, US$5 per day, 20 searches, 25 fetches, 8 council research calls) MUST be enforced and editable.
FR-K-7	Skills MUST load from `agent/skills/*/SKILL.md` and user skill folders; `/` lists commands and skills.
FR-K-8	Durable facts the user states MUST become `propose_record_update` proposals; Kestrel MUST NOT edit records directly.
FR-K-9	Attachments (PDF, DOCX, CSV, images) MUST be readable and treated as untrusted data.
FR-K-10	The tool registry MUST be exposed over MCP stdio read-only: tools that change state (`paper_place_order`, `paper_cancel_order`) MUST NOT be exposed.
FR-K-11	Chats MUST support new, clear, delete (with confirmation), rename, pin, search and bulk clear.

Council

ID	Requirement
FR-C-1	Council MUST be explicit (toggle or `/committee`), never automatic.

ID	Requirement
FR-C-2	It MUST run three specialists, one challenge call each and a separate chair, with concurrency two.
FR-C-3	Specialists MUST use role-scoped tools; paid web search/fetch excluded in this release.
FR-C-4	Each assessment MUST validate against the versioned contract (verdict, evidence, against, uncertainty, invalidation).
FR-C-5	Cost MUST be reserved before each call; a quarter of the turn ceiling is held for synthesis; cancellation aborts the tree.
FR-C-6	Ambiguous subjects or unavailable data MUST yield clarification or gaps, never chart-context substitution.

Paper trading

ID	Requirement
FR-P-1	Only Alpaca's **paper** base URL MAY be used; keys MUST be verified against it and live keys refused.
FR-P-2	Keys MUST be stored 0600 in `alpaca-paper.json` (or env) and never sent to the browser or agent.
FR-P-3	The paper book MUST mirror Alpaca fills into `paper/portfolio.json`; manual book edits MUST be refused in PAPER mode.
FR-P-4	Orders: market/limit, buy/sell, day/gtc, whole or fractional shares, or notional USD for market orders; cancel open orders.
FR-P-5	Kestrel tools `paper_account`, `paper_place_order`, `paper_cancel_order` MUST pass the same server gate.
FR-P-6	Approval `ask` (default) MUST create pending tickets; `auto` MUST submit directly; optional limits MUST refuse deterministically with a reason.
FR-P-7	Only the PIN session MAY change paper settings; every order MUST record origin (user/kestrel + thread).

9. Non-functional requirements

Domain 01 Product requirements and scope

Quality	Requirement / target
Performance	Agent overhead before first provider byte p95 ≤ 150 ms; context pack ≤ 500 ms p95 warm; panel open ≤ 100 ms; no N+1 calls to FinSight; idle CPU ≈ 0 (event-driven)
Availability	Terminal works offline from the persisted cache; failed upstream refreshes serve last-known-good with stale flags
Reliability	A crash mid-turn leaves an `interrupted` partial message; the supervisor restarts the agent with backoff; DB integrity check and backup before migrations
Security & privacy	Loopback by default (LAN is an explicit opt-in); PIN; secrets in Keychain or 0600 files; no telemetry; nothing personal in the repo (§20)
Usability	Keyboard command bar, ⌘/ for Kestrel, responsive layout to phone width, installable PWA
Accessibility	Axe checks in the E2E suite, accessible names match visible text, keyboard-operable tabs, reduced-motion support, light mode, high-contrast-friendly tokens
Maintainability	TypeScript strict, erasable syntax for native Node execution, module boundaries, specs before code, ADRs for significant decisions
Portability	macOS primary; Windows/Linux supported for the terminal; Keychain behind an interface with env fallback
Cost	DeepSeek default (≈US$0.0003–0.0008 per simple turn); caps per turn and per day
Scalability	Plugins (brains, skills, strategies, search providers) add capability without core edits; versioned contracts

10. System architecture

`Domain 02 System architecture`

10.1 Context

FinSight is a local system with one human user. It reaches out to keyless market-data sources, the Alpaca paper API, model providers and research sources. Coding agents can attach to its tools over MCP.

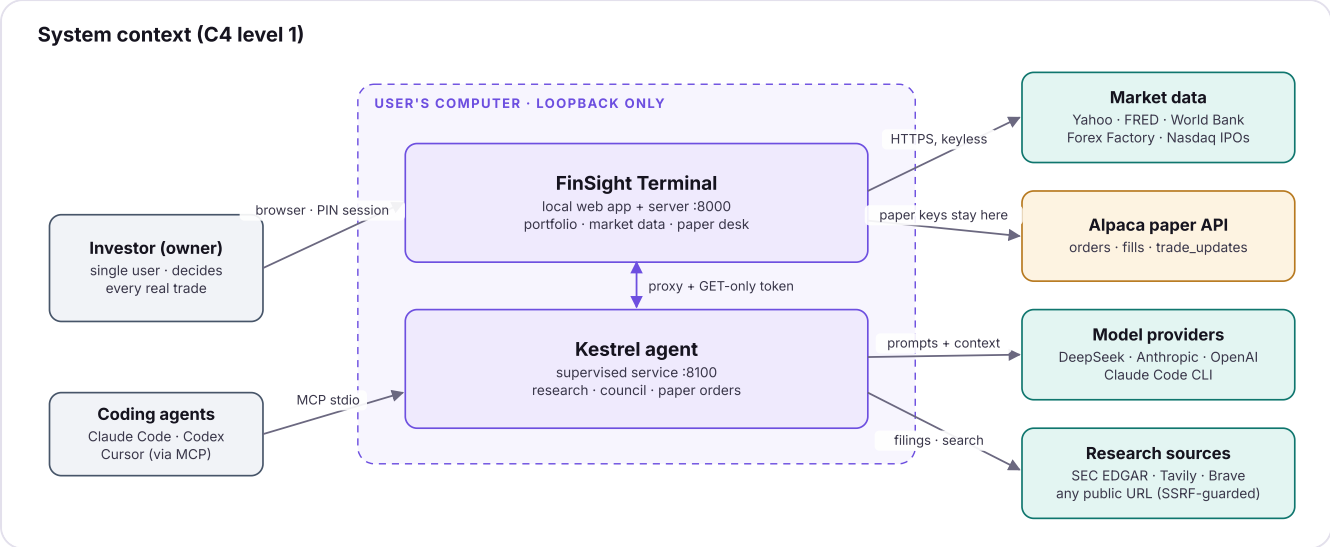


Figure 4 — System context

10.2 Containers and processes

Three processes cooperate on the user's machine: the browser UI, the FinSight server (which also supervises the agent and links to the paper broker) and the Kestrel service. All persistent state lives in `FINSIGHT_DATA_DIR`, outside the repository.

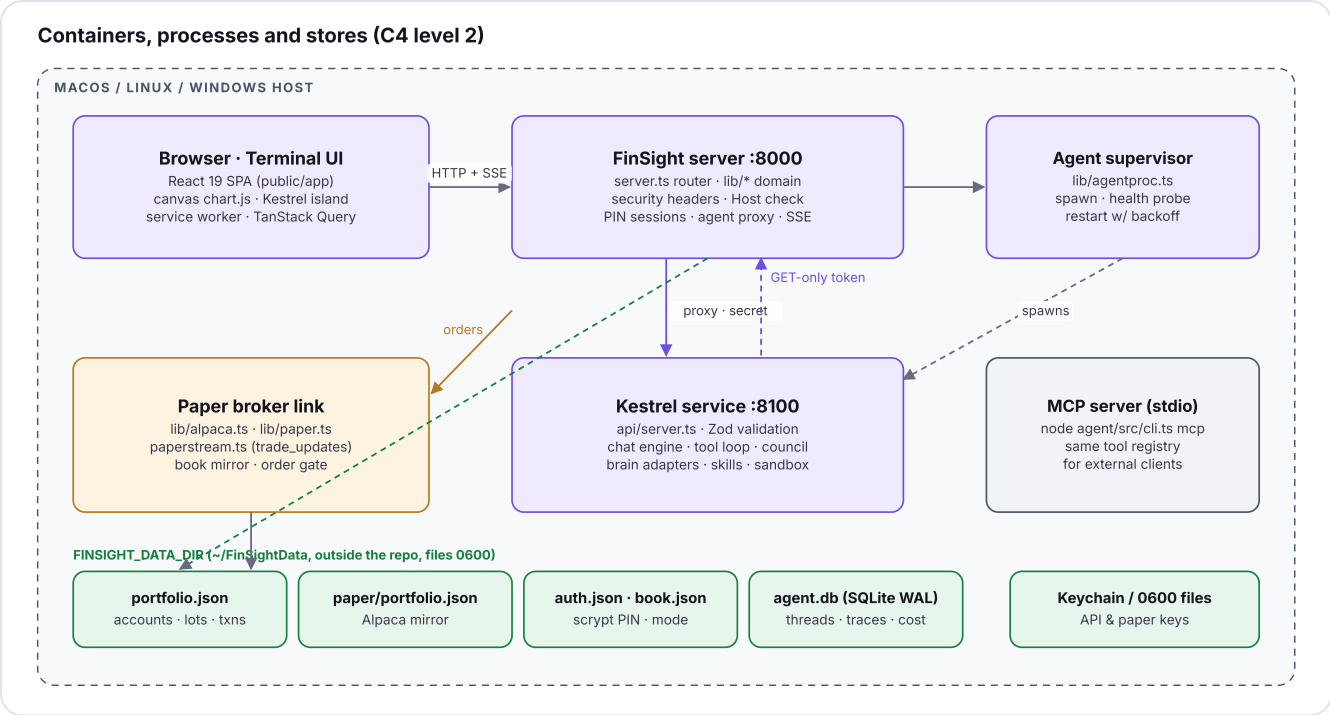


Figure 5 — Containers, processes and stores

Process	Port	Started by	Responsibility
FinSight server	8000 (8003 demo, 8002 standalone)	`node server.ts`, `start.command`, autostart	UI, `/api/*`, sessions, market data, paper link, agent proxy & supervision
Kestrel service	8100 (8103 demo)	FinSight supervisor	Chat engine, tools, council, skills, SQLite store
MCP server	stdio	External client	Exposes the tool registry to Claude Code / Codex / Cursor
Strategy sandbox	child process	Kestrel	Executes strategy engines with timeout and memory caps

10.3 FinSight server components

`server.ts` is only routing, security headers, the session gate and the agent proxy. Domain logic lives in `lib/` modules with single responsibilities.

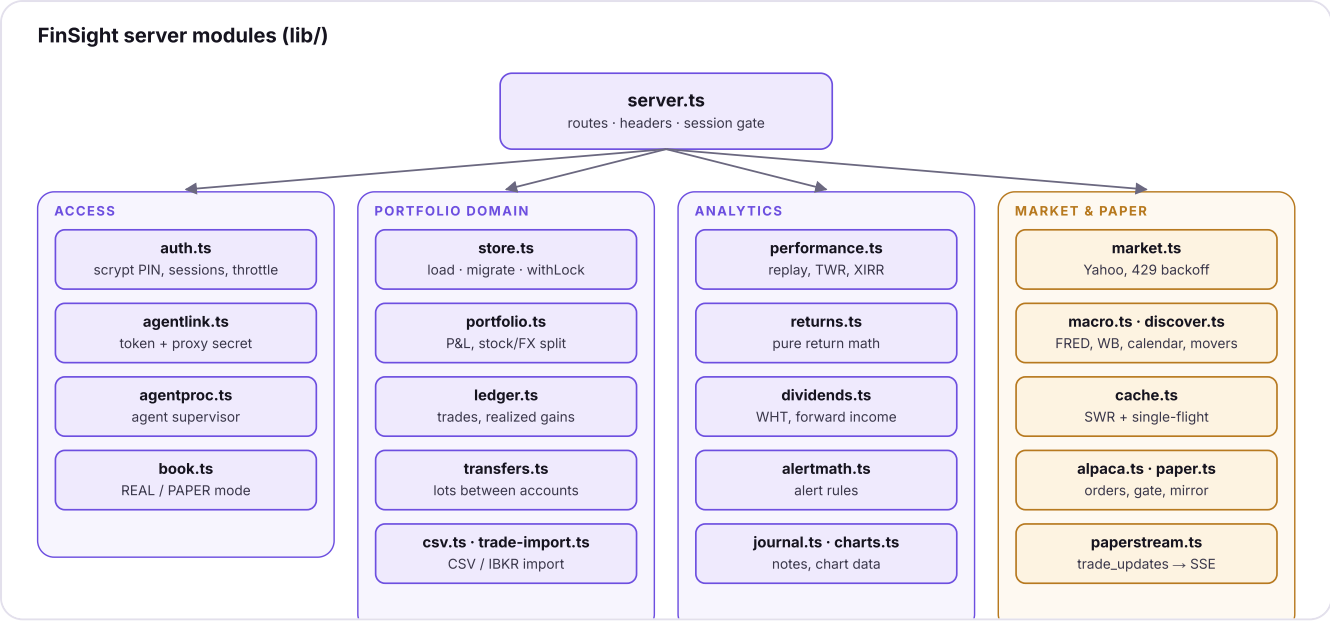


Figure 6 — Server modules

10.4 Kestrel agent components

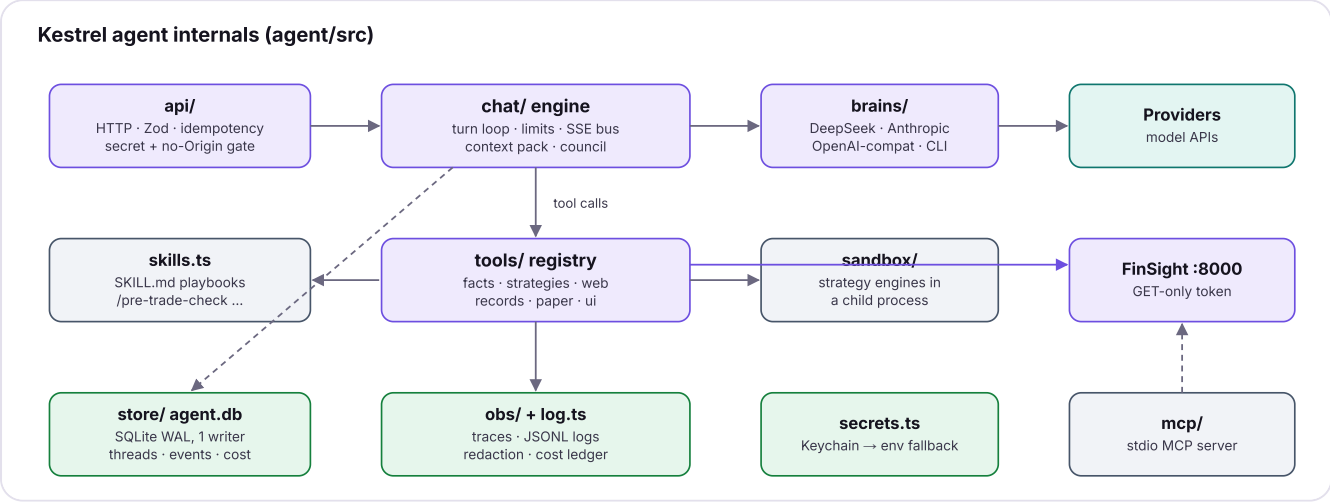


Figure 7 — Kestrel internals

10.5 Key runtime sequences

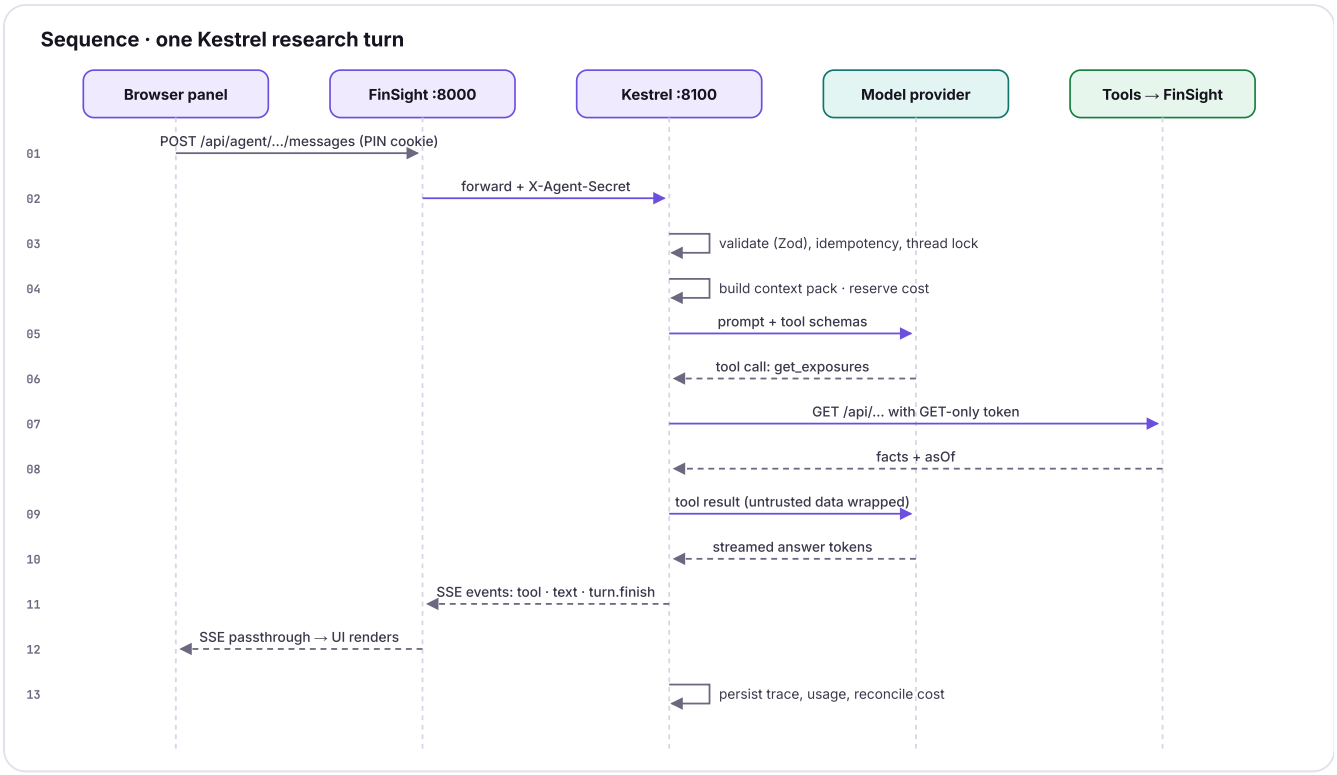


Figure 8 — One Kestrel research turn

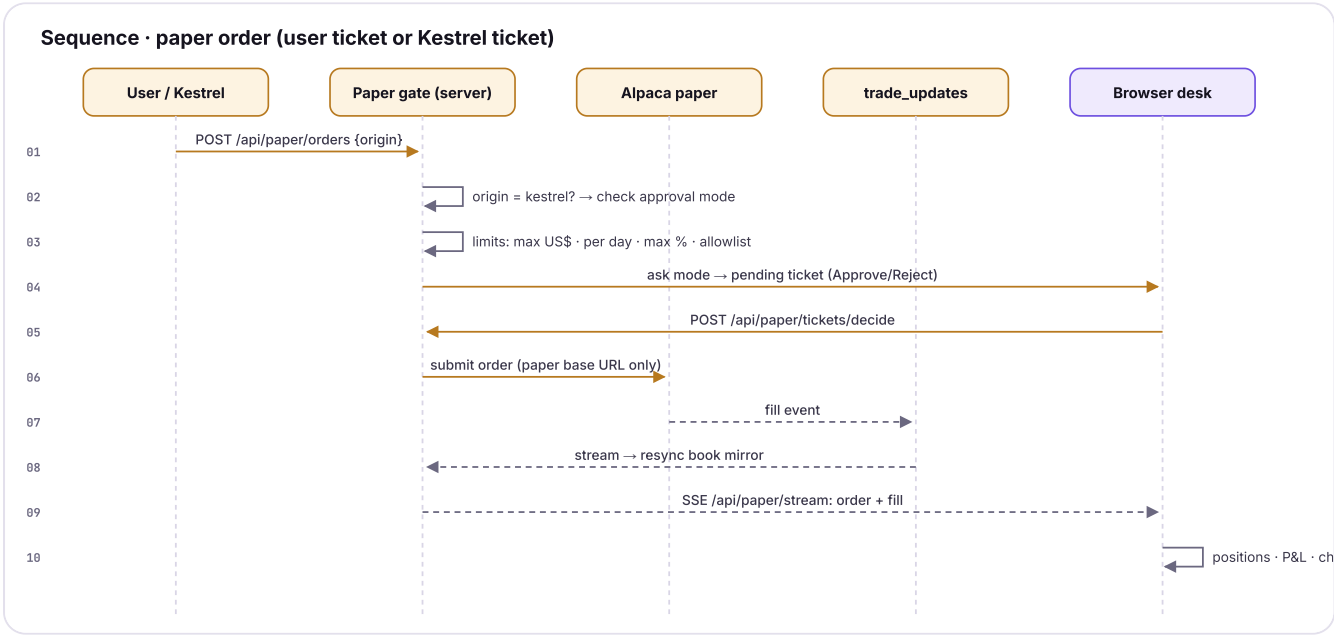


Figure 9 — Paper order through the gate

10.6 Architectural style

FinSight is a **modular monolith split into two local processes**. The FinSight server owns the book, market data and the UI; the Kestrel service owns the conversation, tools and model calls. They are separate so that the agent can crash, restart or be absent without affecting the terminal, and so that the agent's write access to FinSight can be limited to a token with an allowlist. Inside each process the code is layered:

HTTP routing at the edge, domain modules in the middle (`lib/`, `agent/src/chat`, `agent/src/tools`), and storage at the bottom.

10.7 Communication

Path	Style	Why
Browser ↔ FinSight	Synchronous REST/JSON + polling (TanStack Query) + SSE for paper events	Simple, cache-friendly, works offline from the shell
Browser ↔ Kestrel	Through FinSight's `/api/agent/*` proxy; SSE event stream with replay	One origin, one session gate; long answers stream
Kestrel → FinSight	REST with a rotated token; GET-only plus paper order/cancel	Least privilege for the agent
FinSight → Alpaca	REST for orders + `trade_updates` stream	Broker is the source of truth for fills
Background work	In-process timers: alert checks (60 s default), cache sweeps, paper resync (5 min), session janitor	No queue infrastructure needed for one user

10.8 Failure boundaries

- Agent down → the terminal keeps working; the panel explains why and the supervisor restarts the agent with backoff.
- Market source down → cached last-good values with stale flags; no blank panels.
- Broker stream down → the desk shows the stream state and resyncs on reconnect.
- Strategy code fails → contained in a sandboxed child process with time and memory limits.
- A UI panel fails to load → contained by `IslandBoundary`; other panels stay active.

10.9 Key architectural decisions

Decision	Alternatives considered	Reason
Local-only, loopback-bound	Hosted web app	Privacy, zero cost, no accounts
Zero-dependency Node server	Express/Fastify	Small supply-chain surface; nothing to install
JSON files for the book, SQLite for the agent	One database for both	Book is small and human-readable; agent needs transactions, FTS and event replay
Agent as a separate supervised process	Agent inside the server	Fault isolation and least-privilege access
Tools compute, the model explains	Let the model calculate	Correctness and auditability of numbers
Alpaca paper only	Real broker APIs	Safe practice; no real-money path exists in code

Decision	Alternatives considered	Reason
Thinking effort as a per-turn choice	Fixed effort	Recorded in `agent/docs/ADR-THINKING-EFFORT.md`

11. Technology stack

`Domain 03 Technology stack selection`

Layer	Technology	Why
Runtime	Node.js ≥ 24, native TypeScript (erasable syntax)	No build step for the server; one language end to end
Server	`node:http`, native `fetch`, zero runtime dependencies	Small attack surface, nothing to install
Frontend	React 19.3, TypeScript 7, TanStack Query 5, Zustand store	Modern UI with cached queries and simple global state
Bundler	esbuild 0.28 (committed output in `public/app/`)	Fast, reproducible builds; users skip `npm install`
Charting	Custom canvas engine (`chart.js`, `indicators.js`, `strategy.js`)	Full control, offline, no licence constraints
Agent	TypeScript; `@anthropic-ai/sdk`, `@modelcontextprotocol/sdk`, `zod` 4	Model-agnostic adapters, MCP server, validated contracts
Agent storage	SQLite via `better-sqlite3` (WAL)	Durable, transactional, single-file
Documents	`unpdf`, `mammoth`, `@mozilla/readability`, `linkedom`	Read PDFs, Word and web pages safely
Models	DeepSeek Flash/Pro (default), Claude Haiku, Claude Code CLI, GPT-6 Luna/Sol, any OpenAI-compatible	Choice by cost, depth and vision
Data sources	Yahoo Finance, FRED, World Bank, Forex Factory, Nasdaq IPO calendar, SEC EDGAR, Tavily/Brave	Free/keyless where possible
Broker (paper)	Alpaca paper REST + `trade_updates` stream	Free fake-money US equities
Testing	`node:test`, Playwright 1.64, `@axe-core/playwright`	Unit, E2E, accessibility, visual
Secrets	macOS Keychain (`finsight-agent`), env fallback, 0600 files	Keys never in repo or prompts

11.1 Selection criteria

- **One language end to end.** TypeScript on Node 24 (run natively with erasable syntax), React in the browser, TypeScript in the agent.
- **Smallest possible dependency surface.** The server has no runtime npm dependencies; the agent's eight runtime dependencies each solve a problem that is costly to own (SQLite binding, MCP protocol, document parsing, schema validation, a model SDK).
- **Free, keyless data.** Public market and macro sources so the product works without sign-ups.

- **Pluggable vendors.** Models, search providers, skills and strategies are plugins, so no single vendor is required.

11.2 Versions, licences and lock-in

All versions are pinned exactly and locked in `package-lock.json` (root and `agent/`). FinSight is MIT-licensed; its dependencies use permissive licences. Lock-in is limited to two places: Yahoo's unofficial endpoints for market data (isolated in `lib/market.ts`, with a deterministic fake market for tests) and Alpaca for paper trading (isolated in `lib/alpaca.ts`, with a fake broker).

12. Frontend engineering and UX

`Domain 04 Frontend engineering and UX`

The terminal is a single-page application. It is designed to feel like a professional trading terminal: dense, keyboard-driven and fast, while staying accessible and usable on a phone.

12.1 Application structure

- **React 19 + TypeScript** source in `web/src`, one app (`App.tsx`) composed of feature folders (`features/positions`, `features/kestrel`, `features/paper`, ...) loaded as lazy chunks.
- **Build:** esbuild (`scripts/build-web.mjs`) emits committed assets to `public/app/` (bootstrap, main, chunks, manifest) so users need no `npm install` to run.
- **State:** `bootstrap.js` creates `window.FinSight` with a Zustand UI store and refresh bus before first paint; server data via **TanStack Query** with polling suspended while locked or hidden.
- **Chart:** a self-contained canvas engine (`public/chart.js`) plus indicator, strategy and journal helpers in plain JS.
- **Appearance:** tokens on `:root` in `public/styles.css`; `data-theme`, `data-font`, `data-mode` applied before paint.
- **PWA:** service worker precaches the shell from the build manifest; warm cache survives restarts.

12.2 Component architecture and design system

- Each panel is an **island**: a React component in `web/src/features/<name>/` with scoped CSS, registered as a lazy chunk in `islands.ts`. Islands render into the existing panel frame, so layout, drag and keyboard behaviour stay consistent.
- Shared primitives live in `web/src/ui/`: `Money`, `Num`, `Pct`, `Spark`, `Loading`, `Empty`, `ErrorLine`, `PanelBody`, dialogs, toasts and keyboard/focus helpers. Formatting is centralised in `ui/format.ts` with unit tests.
- Design tokens (colours, fonts, spacing) are CSS variables on `:root`; themes switch with `data-theme`, `data-font` and `data-mode` before first paint, so there is no flash.

12.3 State management and data synchronisation

State	Where	Rule
Server data (portfolio, markets, watchlist, alerts, FX risk)	TanStack Query, keyed by base currency and account	Polling every 15–120 s; paused while locked or in a background tab; never copied into Zustand
UI state (selection, layout, panel visibility, mode)	Zustand store on `window.FinSight`	Persisted to `localStorage` only for preferences
Mutations	Shared invalidation map and refresh bus	A trade invalidates every query that depends on it
Live events	SSE (Kestrel, paper)	Resume by event id after a drop

12.4 Navigation and interaction

There is no URL router: the terminal is one screen of movable panels. Navigation is by the **command bar** (`ADD`, `SELL`, `Q SYM`, `ALERT`...), clicking a position to load Security Detail, the top bar (Kestrel, REAL/PAPER, account, base currency, layout) and keyboard shortcuts (⌘/ Kestrel, ⌘. council view, Esc to stop). Panels can be reordered by drag or from the keyboard.

12.5 Forms, validation and errors

- Forms validate as the user types only in fields the user has edited, and show the outcome before saving (the trade sheet previews *Position after* and *New average*; the paper ticket shows estimated price, cost and buying power after the order).
- The server validates again; errors come back as plain sentences (for example "Wrong PIN", or a limit refusal reason) and are shown inline or as a toast.
- Destructive actions (delete chat, remove account) ask first; book edits can be undone (25 steps).

12.6 Loading, empty, error and offline states

Every panel uses the shared `Loading`, `Empty` and `ErrorLine` primitives. Stale market data is labelled with its as-of time; a panel whose source failed says so instead of showing zeros. While the screen is locked, API calls fail fast without touching the network. The service worker caches only the static shell (never API responses), so the terminal opens instantly and shows a clear state when the server is unreachable.

12.7 Accessibility

- Automated axe-core checks in Playwright for the terminal, Kestrel and the activity panel.
- Accessible names match visible text; tabs, dialogs and the command menu are keyboard-operable with managed focus (`modalFocus.ts`).
- Reduced-motion support for animated views (council motion, ticker); light mode and high-contrast-friendly tokens.
- UNPEEK hides every money figure for screen sharing.

12.8 Responsive design and browser support

The layout reflows from a two-column desktop terminal to a single column on phones (tested at 390 px); the paper ticket stacks below the chart. FinSight targets current Chromium, Safari and Firefox and installs as a PWA.

12.9 Rendering, SEO and internationalisation

Client-side rendering is the right choice for a private, PIN-locked app; SEO does not apply. The UI is in English. Numbers and money are formatted for the selected base currency; market times use each exchange's time zone (for example `America/New_York` for US sessions) via `Intl.DateTimeFormat`.

12.10 Frontend performance

esbuild produces content-hashed bundles split into lazy chunks per panel; the chart is a canvas engine that draws thousands of candles without a DOM node per bar; polling stops in background tabs. A chunk-retry guard (`shell/chunkGuard.ts`) recovers when a deploy replaces chunks under an open tab.

PART B

Application logic and data

Domains 05–08: backend, data, APIs and integration, identity and access

13. Backend engineering

`Domain 05 Backend engineering`

The backend is the FinSight server (`server.ts` + `lib/`) and the Kestrel service (`agent/src`). Both are plain Node 24 TypeScript with no web framework.

13.1 Responsibilities

Layer	FinSight server	Kestrel service
Edge / controller	`server.ts` : Host check, security headers, session gate, same-origin check for writes, routing, proxies	`api/server.ts` : shared-secret check, `/v1` routes, Zod request schemas, SSE
Domain / service	`lib/portfolio.ts` , `returns.ts` , `performance.ts` , `ledger.ts` , `dividends.ts` , `transfers.ts` , `paper.ts` , `discover.ts` , `macro.ts`	`chat/engine.ts` , `context.ts` , `council-.ts` , `tools/` , `skills.ts` , `sandbox/host.ts`
Integration	`market.ts` (Yahoo), `macro.ts` (FRED, World Bank, Forex Factory), `alpaca.ts` , `agentproc.ts`	`brains/*` (DeepSeek, Anthropic, OpenAI-compatible, Claude Code), `tools/web.ts` , EDGAR
Repository	`lib/store.ts` : locked, atomic JSON persistence; `cache.ts`	`store/db.ts` : SQLite with migrations

13.2 Request lifecycle

1. Set `X-Frame-Options: DENY` and `X-Content-Type-Options: nosniff` .
2. Reject requests whose Host is not this machine (DNS-rebinding defence).
3. Serve static files (ETag, hashed assets) or continue to `/api/*` .
4. Check the session cookie for data APIs; check same-origin for writes.
5. Read the body (Buffers joined before decoding; 25 MB cap for broker statements), validate, call the domain module.
6. Return JSON with `Cache-Control: no-store` ; map errors to 400 (input), 401/403 (auth), 409 (conflict), 429 (throttle) or 502 (upstream).

13.3 Validation and serialisation

The server validates each route's inputs explicitly (symbols, quantities, currencies, PIN format, book names, paper order fields and limits). The agent validates every request, tool input and config file against Zod schemas, and the council output against a versioned contract. Responses are plain JSON; money values are numbers with currency fields, formatted only in the UI.

13.4 Error handling

Domain errors carry a user-readable message and optional status. Upstream failures never leak stack traces to the UI. Unhandled failures in background jobs (for example alert evaluation) are logged and retried on the next tick instead of crashing the process.

13.5 Configuration

Configuration comes from environment variables (listed in §30), the data folder (`config.json` for the agent, `alpaca-paper.json` for paper keys) and macOS Keychain for secrets. Defaults are safe: loopback host, paper approval `ask`, conservative agent limits.

13.6 Background jobs and scheduled tasks

Job	Interval	Module
Price alert evaluation and desktop notification	60 s (min 15 s, `ALERT_INTERVAL`)	`server.ts`
Market cache sweep	hourly	`lib/cache.ts`
Session and login-throttle janitor	hourly	`lib/auth.ts`
Paper book resync	every 5 min and on every fill	`lib/paper.ts`
SSE keep-alive ping	25 s	`server.ts`
Agent supervision (start, restart with backoff)	event-driven	`lib/agentproc.ts`

13.7 Resource cleanup

Timers that should not keep the process alive are `unref()`'d. On SIGINT, SIGTERM or SIGHUP the server shuts down gracefully and stops the agent it started. The agent drains running turns, closes connections and closes the database on shutdown. Long-lived maps (sessions, throttle entries, cache) are swept so a process that runs for months does not grow without bound.

14. Database and data architecture

`Domain 06 Database and data architecture`

FinSight stores two kinds of data with different needs, so it uses two storage technologies.

Store	Technology	Why
Portfolio book, settings, paper mirror	JSON documents in `FINSIGHT_DATA_DIR`	Small, human-readable, easy to back up and inspect; a single writer with a lock is enough
Agent conversations, events, traces, costs, proposals	SQLite (`agent.db`) in WAL mode via better-sqlite3	Transactions, indexes, full-text search, event replay, concurrent readers

14.1 Data model

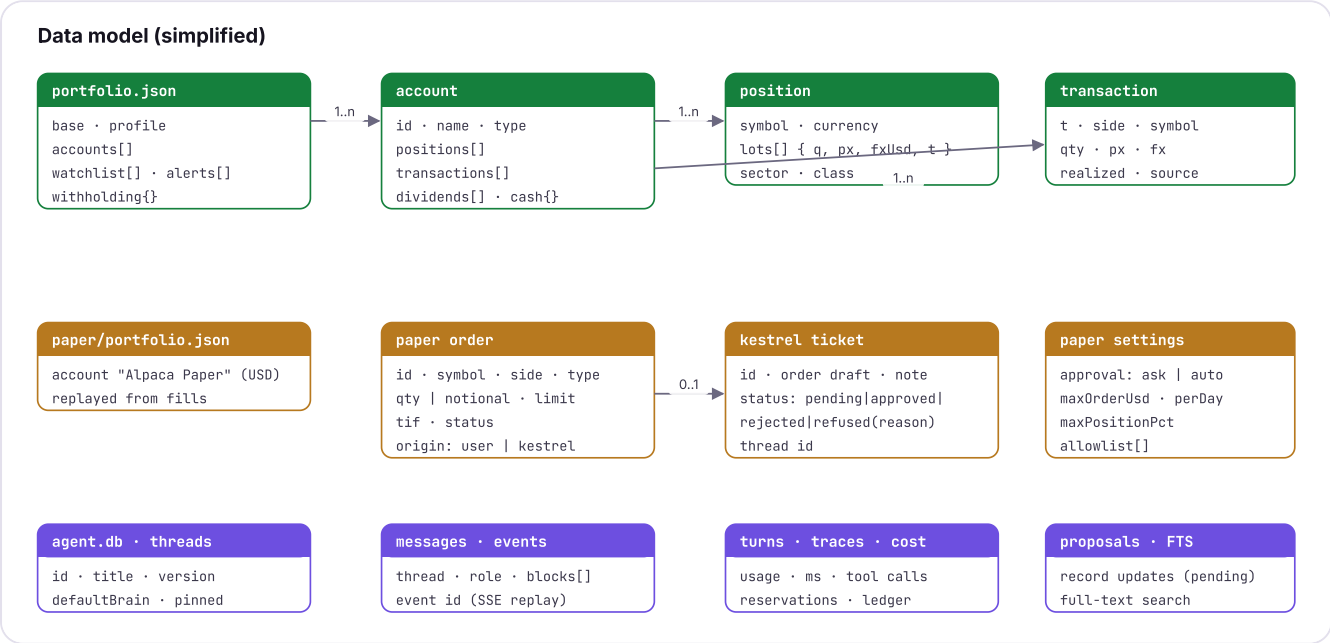


Figure 10 — Data model

File / table	Location	Notes
`portfolio.json`	data dir	Accounts, positions (lots), transactions, dividends, cash, watchlist, alerts; atomic write, 0600
`undo.json`	data dir	25 snapshots for undo
`cache.json`	data dir	Persisted market cache (≤ 30 days)
`auth.json`	data dir	scrypt PIN hash + salt
`book.json`	data dir	REAL / PAPER mode
`paper/portfolio.json`	data dir	Alpaca mirror book

File / table	Location	Notes
`alpaca-paper.json`	data dir	Paper keys, 0600, never served
`agent-link.json`	data dir	Rotated GET-only token and proxy secret
`agent/agent.db`	data dir	SQLite WAL: threads, messages, events, turns, traces, cost ledger, proposals, FTS
`agent/logs/*.jsonl`	data dir	Interaction log, redacted, 14-day retention
`records/`	data dir	INVESTMENT_POLICY.md, INVESTOR_PROFILE.md and other user records

14.2 Keys, constraints and integrity

- Accounts, threads, messages, tickets and orders have generated ids; paper orders carry a `client_order_id` that links each fill to its origin.
- Book invariants are enforced in code: quantities stay non-negative, a sell realises P&L against existing lots, and unrealized P&L always equals stock P&L plus FX P&L.
- SQLite tables use primary keys and foreign keys between threads, messages, turns and events; FTS tables index messages and records for search.

14.3 Transactions and consistency

Every book mutation is a read-modify-write under one in-process lock, written to a temporary file and renamed into place, so a crash leaves either the old or the new file. Multi-row agent writes (a message plus its events and cost) run in one SQLite transaction. Thread updates use an optimistic `version`; a stale version gets HTTP 409. Details in §21.

14.4 Migrations and schema evolution

The agent database has a `schema_migrations` table. Before applying a migration to an existing database, the store copies it to `agent.db.bak-v<N>`, refuses to migrate while another process holds the database, and records each applied version. JSON files are versioned in code and upgraded on load.

14.5 Retention, archival and deletion

Data	Retention
Undo history	Last 25 snapshots
Persisted market cache	Entries older than 30 days dropped
Agent audit logs (`logs/*.jsonl`)	14 days by default (`logRetentionDays` , 1–365)
Chats	Until the user deletes them (`/delete` , bulk clear)
Portfolio, records, paper mirror	Kept until the user removes them

14.6 Backup, recovery and data quality

- A corrupt `portfolio.json` is moved aside to a `.corrupt.json` file rather than overwritten, so it can be restored.
- The agent database is backed up automatically before each migration.
- Full backup is a copy of the data folder (`portfolio.json` , `undo.json` , `agent/agent.db` , `records/`); restore is copying it back with the server stopped.
- Market data quality: every quote and series carries its source and as-of time; stale and failed states are shown, not hidden.

14.7 Multi-tenancy

Not applicable: there is one user per installation. Separate installations (personal, demo, test) use separate data folders.

15. APIs and system integration

`Domain 07 APIs and system integration`

FinSight exposes two HTTP APIs on loopback and integrates with several external services.

15.1 API styles and contracts

- **FinSight `/api/*` — REST/JSON for the browser; SSE for paper order events. Writes require the PIN session and a same-origin request.
- **Kestrel `/v1/*` — versioned REST/JSON with Zod-validated bodies, reachable only through FinSight's proxy with a shared secret; SSE for turn events.
- **MCP (stdio)** — the agent's tool registry exposed to Claude Code, Codex or Cursor, read-only: tools marked as writing (the paper order tools) are filtered out, and a test enforces it.

15.2 HTTP API (selected)

Method & path	Purpose
GET /api/portfolio?base=	Portfolio with quotes, P&L, stock/FX split
POST /api/portfolio/buy · sell · …	Book mutations (session + same-origin, locked, undoable)
GET /api/history · /api/history/batch	OHLCV bars (cached)
GET /api/discover?account=	Movers, earnings, exclusions with basis and asOf
GET /api/events	Earnings and ex-dividend dates
GET /api/agent-status · /api/agent/*	Agent health; session-gated proxy to Kestrel
GET /api/paper/state · settings · stream	Paper account, settings, SSE order/fill stream
POST /api/paper/orders · orders/cancel	Place / cancel paper orders (gate applies)
POST /api/paper/tickets/decide	Approve or reject a Kestrel ticket
POST /api/paper/connect · disconnect · settings · sync	Key management, settings (PIN session only), resync

The full endpoint reference is in `DOCS.md`. Kestrel's own API (`/v1/threads`, `/v1/threads/:id/messages`, `/v1/threads/:id/events`, `/v1/brains`, `/v1/cost`, `/v1/limits`, `/v1/health`) is reachable only through the proxy.

15.3 SSE event model

Kestrel streams typed events (`turn.started`, tool start/finish, text deltas, notices, council stage/assessment events, `turn.finish` with usage and cost). Each event has a monotonically increasing id so a reconnecting client resumes without duplicates. The paper stream (`/api/paper/stream`) carries order and fill events and stream-state changes.

15.4 Idempotency and duplicate prevention

- Kestrel message creation accepts an idempotency key, so a retried POST does not start a second turn.
- Approving a Kestrel paper ticket is claimed under a lock (pending → approving), so a double click or a second tab cannot send the order twice.
- SSE clients resume from `Last-Event-ID`, so reconnects neither lose nor repeat events.

15.5 Pagination, filtering and sorting

Lists are small for a single user, so most endpoints return complete sets and the UI filters and sorts (positions by column, group and search; ledger by filter). Chat lists, history batches and search results are bounded server-side.

15.6 External integrations

Service	Used for	Resilience
Yahoo Finance (unofficial)	Quotes, history, FX, search, news, screens, fundamentals	12 s timeout, host fallback, single-flight cache, stale-while-revalidate, 429 backoff
FRED, World Bank	Macro series and indicators	Cached; panel shows the last good value
Forex Factory, Nasdaq	Economic calendar, IPO calendar	Cached; personal-use terms respected
SEC EDGAR	Filings and financials for Kestrel	Timeouts; content treated as untrusted
Model providers (DeepSeek, Anthropic, OpenAI-compatible)	Kestrel and council reasoning	Per-turn time and cost limits; model can be switched
Search (Tavily, Brave, SearchGPT)	Web research	Per-turn search and fetch limits; SSRF guard on fetch
Alpaca paper	Paper orders, fills, account	Paper base URL constant; stream with resync; fake broker for tests

15.7 Rate limiting and quotas

Login attempts are throttled per IP (after five failures, a delay that grows to five minutes). Kestrel enforces per-turn quotas on steps, seconds, web searches, page fetches and council calls, plus dollar caps per turn and per day. Calls to Yahoo back off on HTTP 429.

15.8 Versioning and compatibility

The agent API is versioned under `/v1`; the council contract and lenses carry version numbers in `shared/council-lenses.ts` and `council-contract.ts`. FinSight's own `/api` is consumed only by its bundled UI, which ships together with the server, so they change in the same commit.

16. Identity and access management

`Domain 08 Identity and access management`

FinSight has exactly one user, so identity is simple by design. Access control is about keeping out other processes, web pages and documents on the same machine, and about limiting what the agent can do.

16.1 Authentication

- A numeric PIN set on first run, hashed with **scrypt** (random salt, run off the event loop) and compared with `timingSafeEqual`. Stored in `auth.json` with 0600 permissions.
- **Throttling:** after five wrong PINs from the same address, further attempts wait 15 s, rising by 15 s per failure up to 5 minutes. Keyed by source address so a third party cannot lock the owner out.
- The lock screen also selects the book (REAL or PAPER) for the session.

16.2 Sessions

- A random session token in an `HttpOnly; SameSite=Strict` cookie.
- Idle expiry after `LOCK_IDLE_MIN` minutes (default 480); the 🔒 button locks immediately; an hourly janitor removes expired sessions.
- When locked, the browser stops calling protected APIs and any 401 returns the UI to the lock screen.

16.3 Authorisation

Principal	Can	Cannot
Owner (PIN session)	Everything: book edits, settings, paper keys, limits, approvals	—
Kestrel (rotated agent token)	Read portfolio facts; place and cancel **paper** orders through the gate	Edit the real book, change settings, limits, approval mode or keys
MCP client	Call Kestrel's read-only tools	Place or cancel paper orders (those tools are not exposed), or write anything else
Unauthenticated	Load the UI shell and the lock screen	Read or change any data

The agent token and the proxy secret are random values rotated at every server start and stored in `agent-link.json` (0600).

16.4 Account lifecycle and recovery

There is no registration, OAuth, MFA or tenant model: they would add risk without benefit for one local user. To reset a forgotten PIN, the owner deletes `auth.json` in the data folder (holdings are kept) and sets a new PIN on next launch.

16.5 Audit of sensitive actions

Every paper order records its origin (user or Kestrel, with the chat thread), and every Kestrel tool call is in the audit log with its turn id. Record changes proposed by Kestrel are stored as proposals until the owner confirms them.

PART C

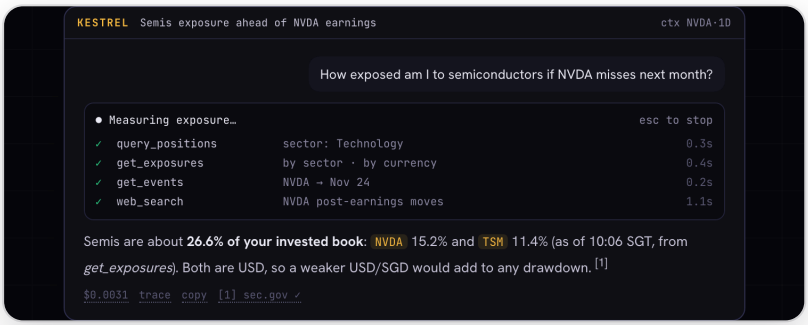
AI engineering and agentic systems

Domain 24: the Kestrel agent, the investment council and agentic paper trading

17. Kestrel agent design

`Domain 24 AI engineering and agentic systems`

Kestrel is FinSight's AI research agent and the centre of its AI engineering. This chapter covers how it is designed; the council and paper trading follow.



`GUIDE RECONSTRUCTION` Screen 16 — A Kestrel answer: tool calls first, then figures with an as-of time, sources and the answer's cost

17.1 Principles

1. **Facts from code, words from the model.** Deterministic tools produce every figure; the model explains, compares and cites.
2. **Show the work.** Tool log, sources, asOf, cost and trace are part of every answer.
3. **Read-only toward the real world.** No tool can trade real money, move funds, message anyone or edit records; facts become proposals.
4. **Untrusted inputs stay data.** Web pages, filings and uploads are wrapped as untrusted and cannot issue instructions.
5. **Bounded.** Step, time, search, fetch and dollar limits per turn and per day.
6. **Model-agnostic.** Brains are plugins; no feature depends on one vendor.

17.2 Turn lifecycle

1. Validate input (Zod), check idempotency key, take the per-thread turn lock.
2. Build the **context pack**: a dated, compact snapshot of holdings, cash, policy and page context (symbol, timeframe, strategy).
3. Order the prompt for **prompt-cache** efficiency (stable system + tools first, volatile context last).
4. Reserve cost against turn and daily caps; call the brain with tool schemas.

5. Execute requested tools with schema validation; shape results with retrieval timestamps; wrap untrusted content.
6. Loop until the model answers or a limit is hit; stream events; persist message, trace, usage; reconcile cost.

17.3 Autonomy levels

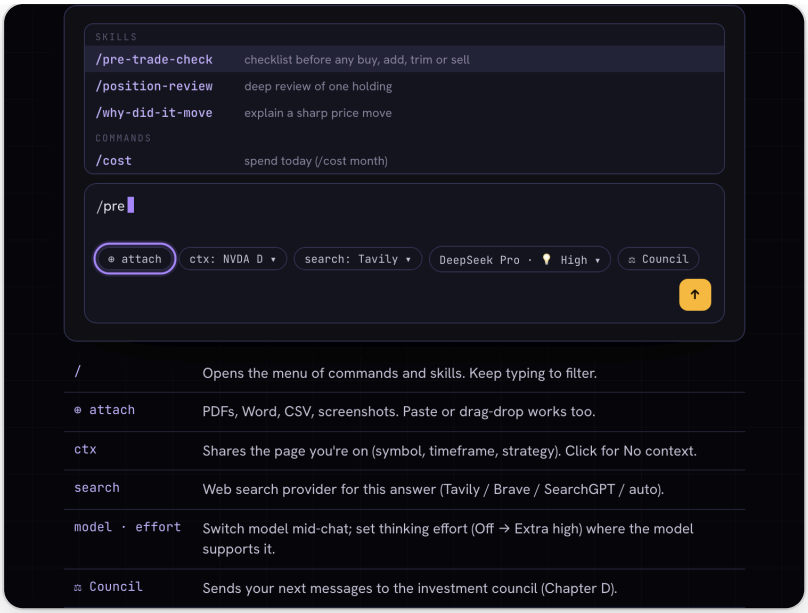
Level	Meaning	Status
L0	Answer in the foreground	Shipped
L1	Background task (durable, budgeted, resumable)	Specified (FR-5)
L2	Scheduled or event-triggered runs	Specified, not built — prerequisite of FEAT-013
L3	Writes proposals the user confirms	Shipped for records; paper orders use the ticket gate

17.4 Limits and cost

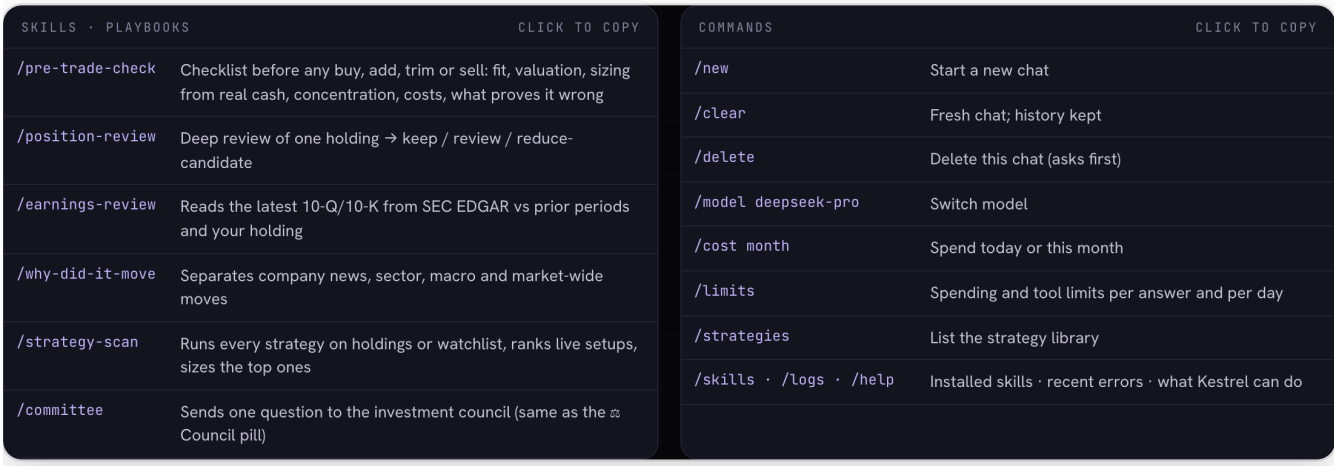
Limit	Default	Range
Steps per turn	25	1–50
Seconds per turn	300	5–900
US$ per turn	0.25	> 0 – 100
US$ per day	5	> 0 – 1000
Searches per turn	20	configurable
Fetches per turn	25	configurable
Council research calls	8	configurable; optional `councilTurnUsd`, `councilOutputTokens`

17.5 Skills (playbooks)

Skill	What it does
/pre-trade-check	Objective fit, valuation, sizing from real cash, concentration, liquidity, costs/taxes, invalidation
/position-review	Why owned, weight and P/L, fundamentals trend, strategies, risks → keep / review / reduce-candidate
/earnings-review	Latest 10-Q/10-K from EDGAR vs prior periods and the holding; what the report changes
/why-did-it-move	Separates company, sector/peer, macro and market-wide drivers; review-trigger check
/strategy-scan	Runs every mechanical strategy on holdings/watchlist, ranks setups, sizes the top ones with event checks



GUIDE RECONSTRUCTION Screen 17 — The composer, control by control: attachments, context, search mode, model and thinking, council and lens



GUIDE RECONSTRUCTION Screen 18 — Commands and skills: `/` opens playbooks such as `/pre-trade-check` and `/why-did-it-move`

17.6 Agent tool registry

Category	Tools
Portfolio facts	`get_portfolio`, `query_positions`, `portfolio_weights`, `get_exposures`, `get_cash`, `get_accounts`, `get_context_pack`
Market & macro	`get_quote`, `get_history`, `get_earnings_dates`, `get_macro_series`, `get_fund_profile`, `valuation_metrics`
Strategies & sizing	`list_strategies`, `get_strategy`, `run_strategy`, `scan_strategies`, `size_position`
Research	`edgar_filings`, `edgar_financials`, `web_search`, `fetch_url`
Records & memory	`get_policy`, `get_profile`, `get_decisions`, `search_memory`, `propose_record_update`

Category	Tools
Attachments	`list_attachments`, `read_attachment`, `search_attachments`
UI	`render_ui` (Intelligent UI views: cards, tables, charts)
Paper (PAPER mode)	`paper_account`, `paper_place_order`, `paper_cancel_order`

17.7 Plugin manifests

Plugin	Location	Defines
Brain	`agent/plugins/brains/<id>/`	Provider, model id, pricing, thinking levels, capabilities (deepseek-flash, deepseek-pro, haiku, claude-code, gpt-6-luna, gpt-6-sol)
Skill	`agent/skills/<name>/SKILL.md`	Agent Skills-format playbook: description, steps, required tools
Strategy	strategy library + `manifest.json`	Card (prose/metadata) and optional mechanical engine run in the sandbox
Search provider	agent config	Tavily, Brave, SearchGPT; chosen per answer

17.8 Model selection and routing

Brains are plugins with a manifest: provider, model id, pricing, cost tier, thinking levels and capabilities (tools, vision, context size). DeepSeek is the default for cost; Claude, OpenAI-compatible models and the Claude Code CLI are alternatives. The user picks the model and thinking effort per chat (`/model`); the council's qualified configuration is DeepSeek Flash with thinking off.

17.9 Prompt and context engineering

- A stable system prompt with the product rules (tools for numbers, cite as-of and sources, untrusted content is data).
- A **context pack**: a dated snapshot of holdings, cash, policy and what is on screen (symbol, timeframe, strategy).
- The user's own records (`INVESTMENT_POLICY.md`, `INVESTOR_PROFILE.md`) and skills when invoked.
- Prompt order optimised for provider prompt caching: stable parts first, volatile context last (FEAT-005).

17.10 Retrieval

Kestrel retrieves by tools rather than a vector index: portfolio tools, SEC EDGAR, web search and fetch, full-text search over past chats and records (`search_memory`) and search inside uploaded files (`search_attachments`).

17.11 Structured output and validation

Tool inputs and outputs are Zod-validated. UI views returned by `render_ui` follow a typed spec. Council assessments must validate against the versioned contract and cite evidence ids; one repair attempt is allowed per stage before the result is marked partial.

17.12 Safety: injection, exfiltration and authorisation

- Fetched pages, filings and uploads are labelled untrusted data and cannot issue instructions.
- `fetch_url` blocks private, loopback, link-local and metadata addresses, pins DNS to the socket and checks redirects.
- No tool can send messages, move money or edit records; durable facts become proposals the user confirms.
- Paper orders are the only write, behind approval mode and limits.

17.13 Evaluation

`agent/evals/council-cases.ts` and `agent/scripts/evaluate-council.ts` run synthetic council cases; `qualify-ui.ts` qualifies Intelligent UI output; agent unit tests cover the tool loop, limits, idempotency and redaction. Evaluations show the pipeline is reliable; they do not claim investment performance.

17.14 Fallbacks and provider outages

If a provider fails, the turn ends with a clear error and the user can switch model in one menu. If Kestrel itself is down, the terminal keeps working and the panel explains how to restart it.

18. Investment council design

Domain 24 AI engineering and agentic systems

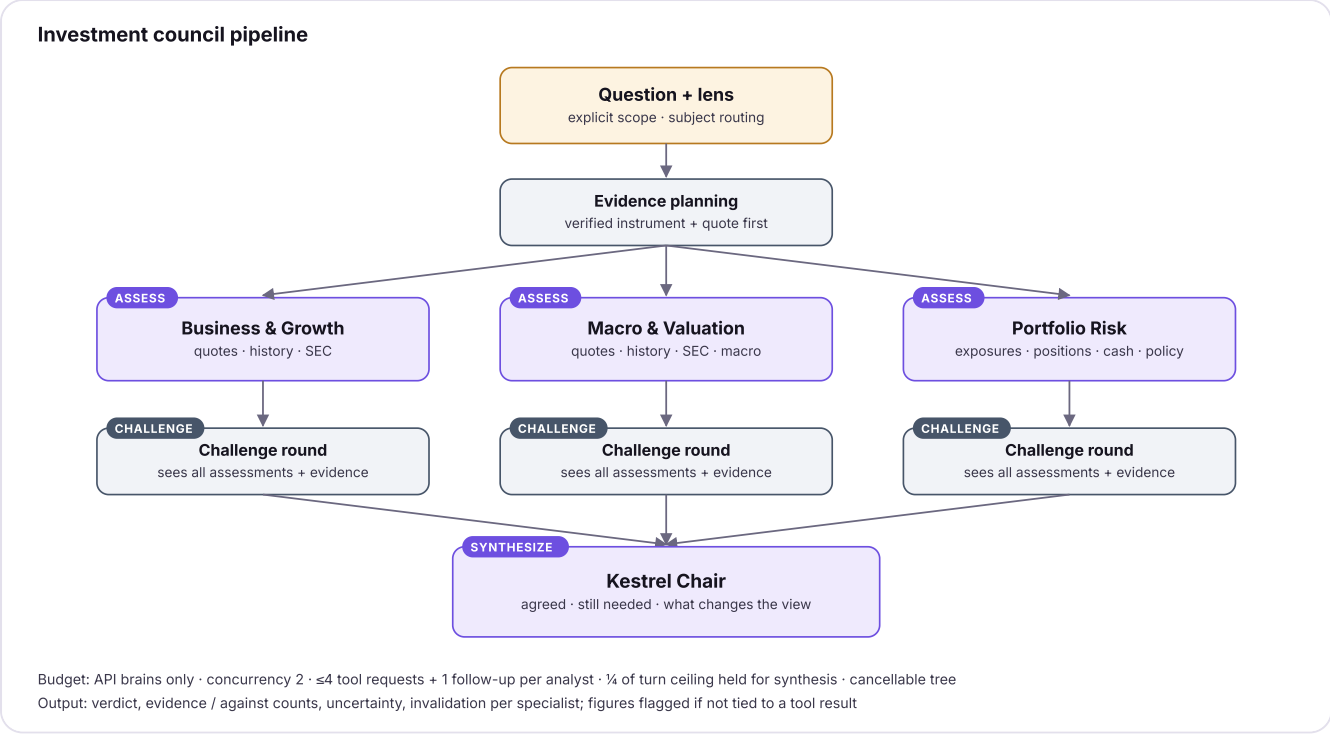


Figure 11 — Council pipeline

Aspect	Design
Roles	Business & Growth · Macro & Valuation · Portfolio Risk · separate Kestrel Chair
Stages	01 Assess (independent histories) → 02 Challenge (sees all assessments and evidence) → 03 Synthesize
Tools per role	Growth/Macro: quotes, histories, SEC filings/financials · Risk: exposures, positions, cash, policy · paid search excluded
Bounds	API brains only; concurrency 2; ≤ 4 tool requests + 1 follow-up per initial analyst; one validation repair per stage; deadline and context bound
Accounting	Conservative reservation before every call in turn and daily budgets; unknown usage keeps the reservation
Exclusivity	No other reply admitted while a council runs; a council is rejected while another turn runs
Cancellation	Aborts the whole tree; late results cannot complete a run; reload labels orphaned children interrupted
Contract	Versioned lenses and assessment schema in `council-contract.ts`; shape and reference integrity validated (not factual truth)
Qualified config	DeepSeek Flash, thinking off; 6/6 synthetic reliability rerun; no investment-performance advantage claimed

Aspect	Design
UI	Cards (verdict, evidence/against, uncertainty, would-change-if, chair row) and Chain of thought lanes; ⌘. switches; compact view in side panel; capsule status



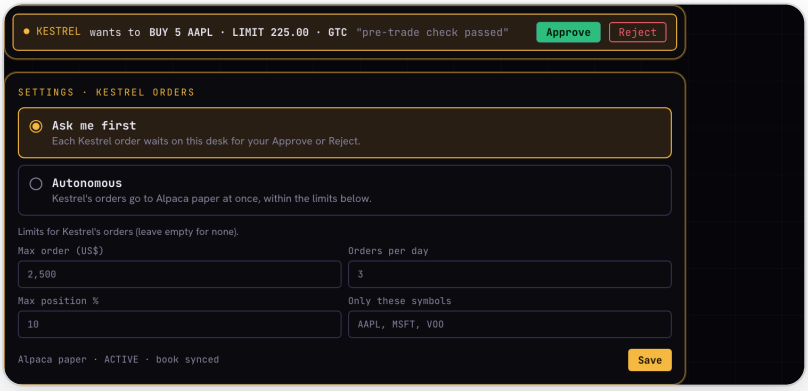
GUIDE RECONSTRUCTION Screen 19 — Chain of thought view (⌘.): what each specialist read, claimed and challenged, and how the chair weighed it

> **Known limitation**
>
> Research readiness stays **partial** while qualitative claim support lacks an independent check. Work item COUNCILQUALITY-001 tracks deeper evidence collection and honest readiness states.

19. Paper trading design

`Domain 24 AI engineering and agentic systems · Domain 07 APIs and system integration`

Concern	Design decision
Broker	Alpaca **paper** API only; base URL is a constant; no live-trading URL exists in code; tests point at a loopback fake (`FINSIGHT_FAKE_MARKET=1`)
Source of truth	Alpaca holds orders, fills, cash and positions; FinSight replays fill activities into `paper/portfolio.json` so every view works unchanged
Live updates	Server keeps `trade_updates` open while keys exist, resyncs on each fill and pushes events over `/api/paper/stream`
Valuation	Prices stay on the existing quote path (Yahoo, 15 s); Alpaca IEX data is not used for valuation
Orders	Market/limit · buy/sell · day/gtc · fractional shares · notional USD for market orders · cancel
Agent access	`paper_account` , `paper_place_order` , `paper_cancel_order` ; every order passes the server gate
Approval	`ask` (default) → pending ticket on the desk; `auto` → submit at once
Limits	Max order US$ · Kestrel orders per day · max position % after order · symbol allowlist; each off unless set; refusal reason returned
Settings authority	Changed only from the terminal (PIN session); the agent can read but not change them
Audit	Every order records origin (`user` / `kestrel` + thread) and appears in Orders, History and the Kestrel tab
Out of scope	Real-money trading, shorts/options in the book, Alpaca market-data streaming, scheduled agent runs, multi-account paper



`GUIDE RECONSTRUCTION` Screen 20 — Kestrel's paper orders: in Ask mode a pending ticket waits for Approve or Reject; the settings tab sets the approval mode and the limits

19.1 Roadmap: agentic trading (FEAT-013, planned)

A daily cycle: trigger → universe → screen (deterministic) → council research → structured `TradeProposal` (size from `size_position`, entry/stop/target, thesis, invalidation) → deterministic gate with new limits (stop required, max loss per trade, max open positions, daily drawdown kill switch, market hours, earnings blackout) → bracket order with idempotency key → manage and score. The Alpaca MCP server may be used for **data only**; its trading toolsets are deliberately not given to Kestrel.

PART D

Security, correctness and system behaviour

Domains 09–13: security, concurrency, performance, reliability, testing

20. Application security

`Domain 09 Application security`

20.1 Assets and trust boundaries

Assets: portfolio and financial data, broker paper keys, model API keys, the PIN/session, the agent token and proxy secret, and the integrity of answers.

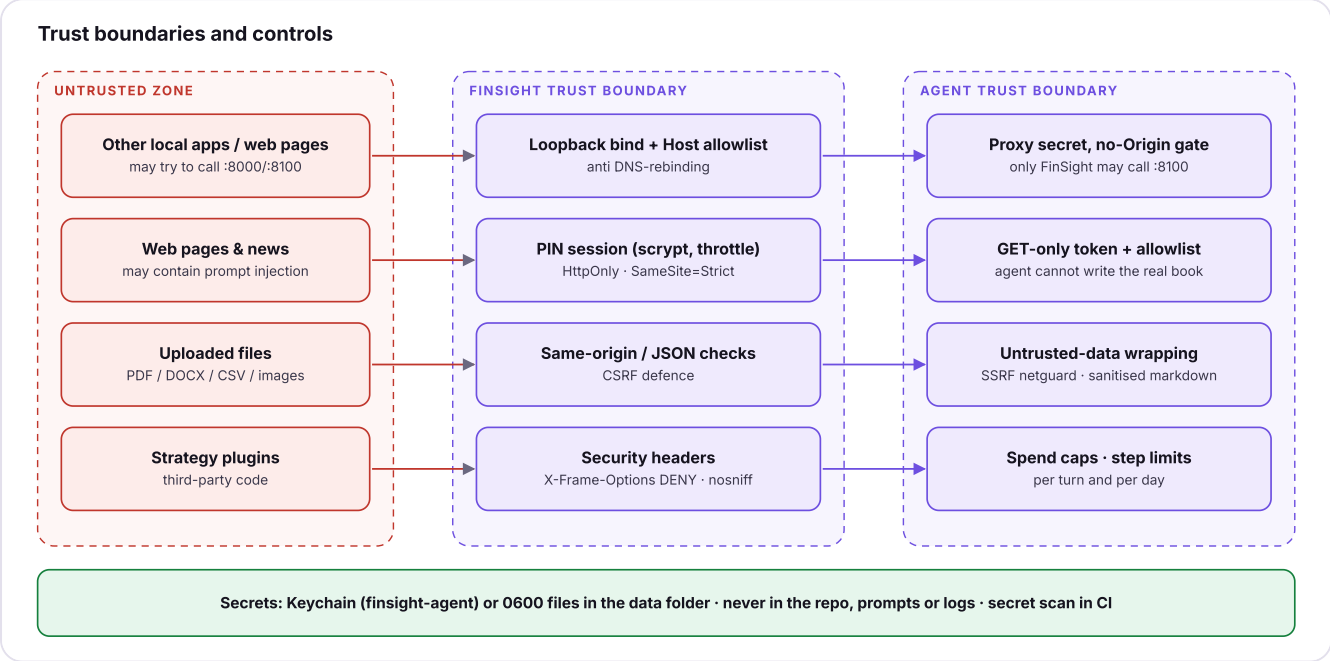


Figure 12 — Trust boundaries and controls

20.2 Threat model (STRIDE summary)

Threat	Example	Control	Verification
Spoofing	Another local process or web page calls :8000/:8100	Loopback bind, Host allowlist (anti DNS rebinding), PIN session, proxy secret, no CORS	Request without secret / wrong Host → 403

Threat	Example	Control	Verification
Tampering	CSRF changes the book	SameSite=Strict, same-origin (`Sec-Fetch-Site`) and Origin checks, JSON content type	Cross-origin POST rejected
Tampering	Agent edits the real book	GET-only token with endpoint allowlist; FinSight sole writer	Token POST → 403
Repudiation	Unclear who placed a paper order	Origin tag + thread on every order; traces and logs	Orders list shows origin
Info disclosure	Keys leak into logs/prompts/repo	Keychain/0600 files, redaction, secret scan in `agent:test` , nothing personal in repo	Seeded secrets absent from logs/DB
Info disclosure	Screen sharing reveals wealth	UNPEEK blanks money figures everywhere	Visual check
Denial of service / cost	Runaway model loop	Step, time, search, fetch and dollar caps; daily cap	Loop test stops at limit
Elevation	Prompt injection in a web page or PDF	Untrusted wrapping, no write/send tools, proposals need confirmation	Injection eval set
Elevation	SSRF via `fetch_url`	Netguard blocks private, loopback, link-local, metadata and redirect-to-private	127.0.0.1, 10/8, 169.254.169.254 blocked
Elevation	Malicious strategy plugin	Child-process sandbox, permission manifest, timeout and memory cap	fs/net/spawn denied
Elevation	Paper agent over-trades	Approval mode, deterministic limits, settings PIN-only, paper URL constant	Limit tests against fake broker
XSS	Model output contains script	Sanitised markdown rendering	Script payloads neutralised
Supply chain	Compromised dependency	Zero runtime deps on server; pinned agent deps; lockfile; minimal surface	CI audit

20.3 Security controls checklist

- Both services bind to `127.0.0.1` by default. The Kestrel service is always loopback-only; FinSight can be bound to the LAN with `HOST=0.0.0.0` , which also turns off the Host check (see below).
- **LAN exception:** `HOST=0.0.0.0` is an explicit opt-in, not a supported security profile. Anyone on the network can reach the lock screen, and the PIN plus throttling is then the only barrier. Use it only on a trusted network.
- **Same-user processes:** 0600 permissions stop other OS users, not other programs running as the owner. Malware running under the owner's account could read the data folder; this is accepted for a personal tool.
- Security headers: `X-Frame-Options: DENY` , `X-Content-Type-Options: nosniff` , Host header check.
- PIN: scrypt KDF off the event loop, brute-force throttling, optional idle lock.

- Agent link: random 32-byte token and proxy secret rotated at each server start (`agent-link.json` , 0600).
- Data files written atomically (temp + rename) with 0600 permissions in a folder outside the repo.
- API keys in macOS Keychain service `finsight-agent` (env fallback); paper keys server-side only.
- Demo and tests use synthetic data, fake market and fake broker — no real data or network.

20.4 Secrets management

Model and search keys are read from macOS Keychain (service `finsight-agent`) first, then environment variables, then a 0600 env file. Paper keys are stored server-side only and never sent to the browser or the agent. Logs are redacted before writing. `agent/scripts/secret-scan.ts` runs as part of `npm run agent:test` and fails the gate if a secret pattern appears in the repository.

20.5 Dependencies and supply chain

The server has no runtime dependencies; the web bundle and the agent use exact pinned versions with lockfiles. The committed web bundle is checked against the source build (`npm run check:web`), so what ships is what was reviewed.

20.6 Hardening backlog

- Add a strict Content-Security-Policy header (defence in depth for rendered content).
- Add a dependency audit to the release gate for the agent and web dependencies.

21. Concurrency and parallelism

`Domain 10 Concurrency and parallelism`

FinSight is single-user but highly concurrent: the browser polls, several tabs may be open, Kestrel reads while the user writes, councils run calls in parallel and broker events arrive asynchronously. The controls below keep state consistent.

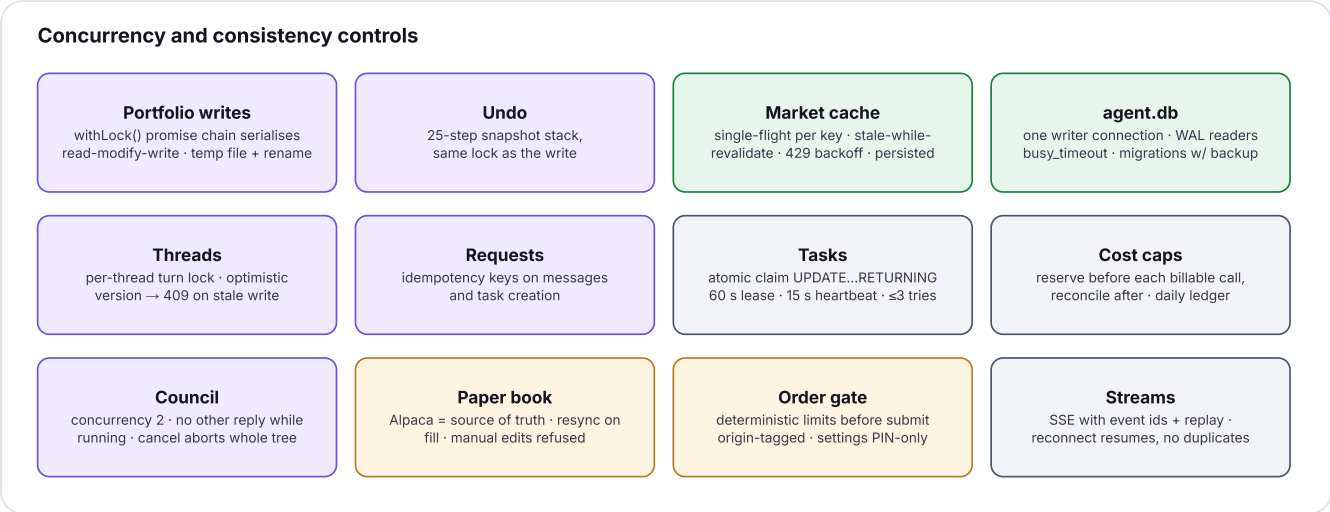


Figure 13 — Concurrency controls

Shared resource	Control	Failure it prevents
`portfolio.json`	`withLock()` promise chain serialises every read-modify-write; atomic temp-file rename	Lost updates from two quick trades or two tabs
Market cache	Single-flight per key; stale-while-revalidate; persisted last-good; 429 backoff	Thundering herd on Yahoo; blank panels during outages
`agent.db`	One writer connection; WAL for concurrent readers; `busy_timeout`	SQLITE_BUSY and torn writes
Chat threads	Per-thread turn lock; optimistic `version`, 409 on stale `expectedVersion`	Two turns interleaving in one thread
Requests	Idempotency keys on message and task creation	Double-submit on retry or flaky network
Background tasks	Atomic `UPDATE … RETURNING` claim; 60 s lease, 15 s heartbeat, ≤ 3 attempts	Two workers running one task; lost tasks after crash
Spend caps	Reserve before each billable call, reconcile after; persisted daily ledger	Parallel calls overshooting a cap
Council	Concurrency 2; exclusive with other turns; cancellation aborts the tree	Budget races, orphaned children
Plugin registry	Immutable snapshot swapped atomically on reload	Half-loaded plugins during a scan

Shared resource	Control	Failure it prevents
Paper book	Alpaca as source of truth; resync on every fill; manual edits refused	Divergent local and broker state
SSE streams	Event ids with replay on reconnect	Missed or duplicated events

21.1 Worked example: the user and Kestrel sell the same position at once

The user clicks **Sell 8 AAPL** on the desk while Kestrel, in Autonomous mode, sends **Sell 8 AAPL** for the same paper position. The figure shows how each layer keeps the result correct.

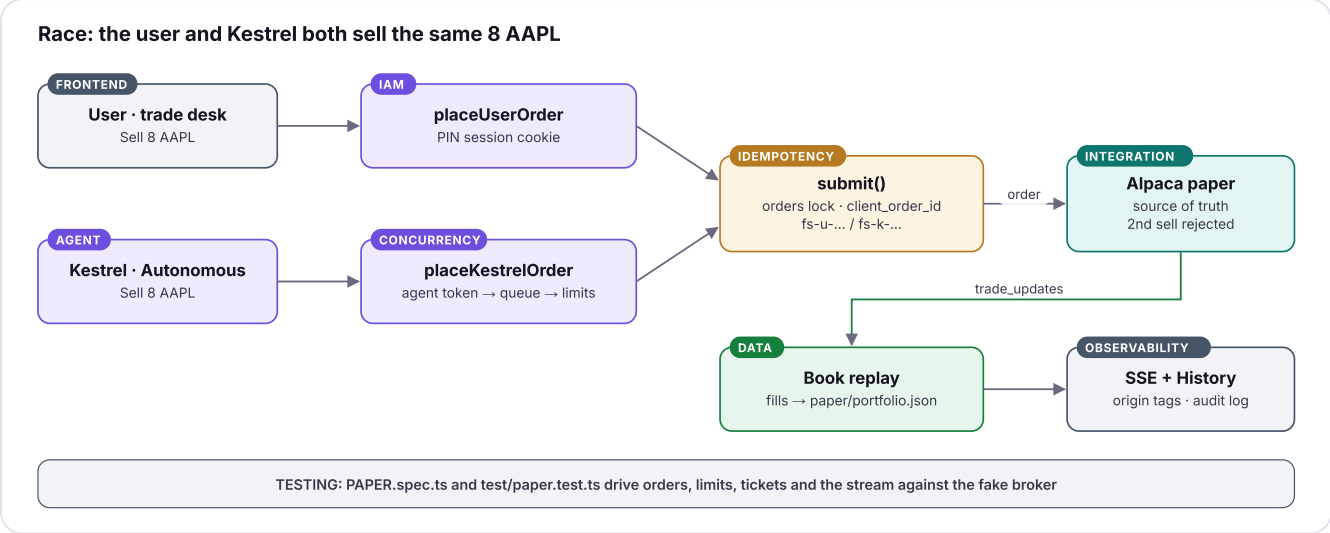


Figure 14 — A concurrent sell from the user and from Kestrel, and the control at each step

Step	What keeps it correct
Both requests arrive	The user's request goes to `placeUserOrder`; the agent-token request to `placeKestrelOrder`
Kestrel's limit checks	Kestrel's orders run one at a time through `kestrelQueue`, so two Kestrel calls cannot both pass the per-day and position checks
Order metadata	Written under the orders lock; each order gets a unique `client_order_id` (`fs-u-…` / `fs-k-…`)
Broker	Alpaca is the source of truth and rejects the second sell for insufficient quantity; the rejection is shown to whoever sent it
Book	Rebuilt by replaying fills from `trade_updates`, so it cannot record a sale that did not fill
After a dropped stream	Reconnect plus `/api/paper/sync` reconciles the book with Alpaca
Audit	Both orders keep their origin, visible in History and the Kestrel tab

22. Performance and scalability

`Domain 11 Performance and scalability`

FinSight runs for one user on one machine, so performance work targets latency and resource use, not throughput. Scale-out techniques are deliberately not used.

22.1 Targets

Measure	Target
Agent overhead before the provider's first byte	p95 ≤ 150 ms
Context pack build (warm)	p95 ≤ 500 ms
Kestrel panel open	≤ 100 ms
Idle CPU	≈ 0 (event-driven; polling stops in background tabs)
Calls from the agent to FinSight	No N+1 patterns: batch endpoints such as `/api/history/batch`

22.2 Caching

Layer	What	Invalidation
Browser shell	Service worker precaches static files and hashed chunks	Cache name bumped when the file set changes; old caches deleted on activate
Static files	ETag on every asset; hashed chunk names	New hash per build
Server market cache	Per-key TTL with stale-while-revalidate, single-flight, persisted last-good (≤ 30 days)	TTL expiry; background refresh
Query cache (browser)	TanStack Query per base and account	Invalidation map after mutations; polling 15–120 s
Model prompt cache	Provider-side caching of the stable prompt prefix	Prompt ordered so volatile parts come last

22.3 Bottlenecks and how they are handled

- **Network to data sources** — the dominant cost; handled by caching, batching and single-flight.
- **Model latency** — streamed to the UI so the first tokens arrive quickly; limits stop long loops.
- **CPU-heavy strategy runs** — moved to a sandboxed child process so the event loop stays free.
- **Rendering** — canvas chart, lazy panels, no DOM per candle.

22.4 Scalability

Load balancing, autoscaling, CDNs and horizontal scaling do not apply to a single-user local app. FinSight scales in **capability** instead: new models, skills, strategies and search providers are added as plugins without changing the core.

22.5 Performance testing

`web/e2e/perf.spec.ts` measures cold and warm loads at desktop and phone sizes, and `startup.spec.ts` checks start-up behaviour; `agent/scripts/cache-probe.ts` measures prompt-cache hits. Load, stress and soak tests are not needed at this scale.

23. Reliability and fault tolerance

`Domain 12 Reliability and fault tolerance`

FinSight should be available whenever the computer is on, never lose a committed change, and degrade visibly rather than fail silently.

23.1 Availability and recovery targets

Target	Value
Availability	Terminal up while the Mac is on (launchd restarts the server if it crashes)
Data loss on a crash (RPO)	None beyond the last in-flight write (atomic writes); agent DB backed up before migrations
Data loss on disk failure, deletion or a lost laptop	Back to the owner's last backup of the data folder. A full restore has not yet been rehearsed (engineering backlog P2)
Recovery time (RTO)	Minutes: restart the server, or copy the data folder back from backup

23.2 Health checks and supervision

- `/api/health` (FinSight) and `/v1/health` (Kestrel); `/api/agent-status` reports the supervisor state; `kestrel doctor` checks the agent's configuration.
- The FinSight server starts the agent, restarts it with backoff if it exits, and stops restarting after repeated crashes within 10 minutes, with a message pointing to the log.
- launchd (`install-autostart.sh`) starts the server at login and restarts it if it crashes, so background price alerts keep working.

23.3 Failure handling by situation

Situation	Behaviour
Yahoo / FRED outage or 429	Serve last-known-good from cache with stale flag; back off and retry in the background
Agent not installed / disabled / crashing	Panel shows the exact reason and fix (`npm run agent:install` , AGENT_AUTOSTART, log path); supervisor restarts with backoff
FinSight unreachable from agent	`FINSIGHT_DOWN` tool error; the model tells the user; nothing fabricated
Turn interrupted (crash, stop)	Partial message persisted and marked interrupted/stopped
Limit reached	Turn ends with a clear notice; `/limits` to adjust
Council subject ambiguous / data missing	Clarification or explicit gap; partial readiness

Situation	Behaviour
Paper key invalid or live	Refused before saving with a message
Paper limit breached	Order refused before reaching Alpaca with the reason
Paper stream drops	Status shown on the account strip; resync on reconnect
Corrupt or old data file	Migrations on load; undo snapshots; DB integrity check and backup before migrations

23.4 Single points of failure

The machine itself and the data folder are the single points of failure. This is accepted for a personal tool and mitigated by the owner's normal backups (Time Machine or an encrypted copy of the data folder) and by the fact that broker records remain the external source of truth for the real book.

24. Testing and quality assurance

`Domain 13 Testing and quality assurance`

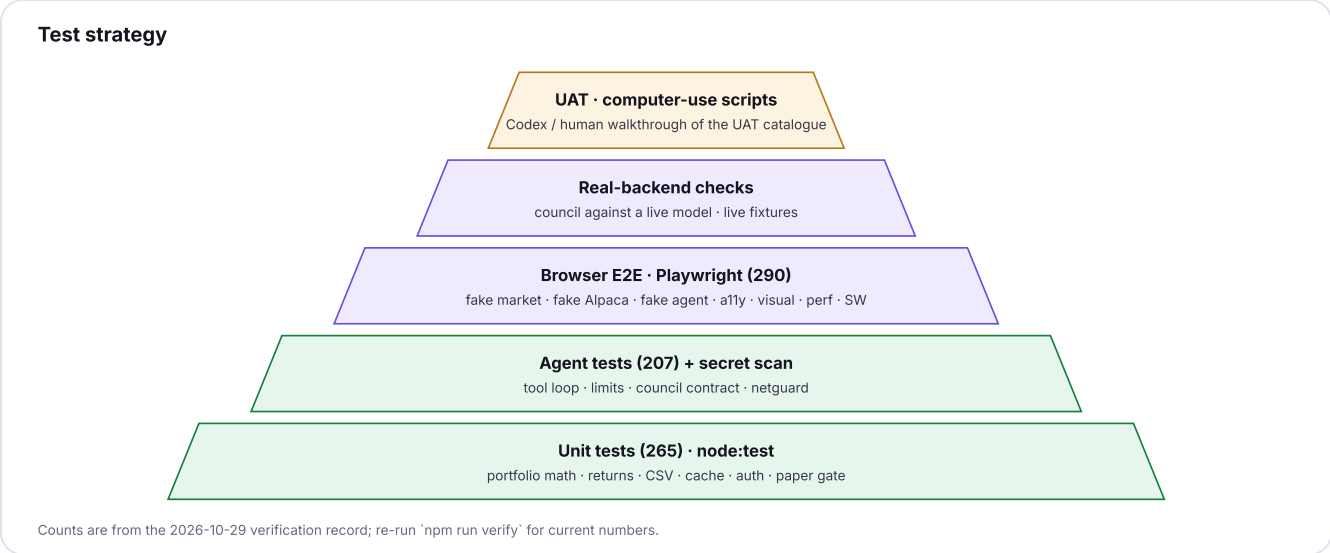


Figure 15 — Test pyramid

Level	Scope	Tooling / fixtures
Unit	Portfolio math, returns, FX split, CSV/IBKR parsing, cache, auth, paper gate and mirror	`node:test`, pure functions, fixtures in `test/fixtures`
Agent	Tool loop, limits, idempotency, council contract and orchestration, netguard, redaction	`node --test`, provider fakes, secret scan
Browser E2E	Every panel and flow, Kestrel streaming, council motion, paper desk, discover, layout drag	Playwright, `FINSIGHT_FAKE_MARKET=1`, `fake-agent.mjs`, fake Alpaca
Accessibility	Axe rules on terminal and Kestrel	`@axe-core/playwright`
Visual	Summary, positions, markets, detail at 1600/390 px, dark and light	Playwright snapshots
Performance & SW	Startup, perf budgets, service-worker upgrade and release	Dedicated specs
Real backend	Council against a live model on synthetic cases	Separate Playwright config
UAT	End-to-end acceptance by a human or Codex computer use	§25 catalogue

24.1 Test data, fixtures and fakes

- **Fake market** (`FINSIGHT_FAKE_MARKET=1`): deterministic quotes, history, FX, news and calendars from `test/fixtures/market`.

- **Fake Alpaca** (`lib/fakealpaca.ts`): accepts PK-prefixed test keys, fills orders and emits `trade_updates` .
- **Fake agent** (`web/e2e/fake-agent.mjs`) and a deterministic council brain for the real-backend suite.
- **Demo portfolio** (`scripts/demo.ts`): nine synthetic holdings in SGD and USD; `demo-safety` tests stop it from touching real data.

24.2 Test counts

Suite	Size
Unit and integration (`test/` , `web/src/**/*.test.ts`)	265 tests in 55 files — all passed
Agent (`agent/test`) + secret scan	207 tests in 23 files — all passed
Playwright E2E (`web/e2e` , default config)	290 tests in 68 spec files (listed)
UAT catalogue	30 cases (results are not recorded in this document)

Unit and agent counts come from runs on 1 November 2026. The Playwright count is from `playwright test --list` ; the E2E suite was not re-run for this revision.

24.3 Release gate

`npm run verify` runs the typecheck, unit tests, the web bundle check and the E2E suite; `npm run agent:test` runs the agent's typecheck, tests and secret scan. Both must pass before a merge to `main` .

25. UAT plan for computer-use testing

`Domain 13 Testing and quality assurance`

25.1 Environment

1. Use a disposable data folder and the offline fakes: `FINSIGHT_DATA_DIR=<tmp> FINSIGHT_FAKE_MARKET=1 NOTIFY=0 PORT=8031 node server.ts` (or `npm run demo` for the seeded book, PIN 2468).
2. For paper tests with the fake broker, connect with test key id `PKE2ETEST1234567890` and the matching E2E secret from `web/e2e/PAPER.spec.ts` (fake broker accepts PK-prefixed ids).
3. For Kestrel tests without a model key, run `node web/e2e/fake-agent.mjs` and set `AGENT_URL` to it; for council tests use a configured DeepSeek key.
4. Viewport 1600×1000 for desktop cases, 390×844 for mobile cases. Record a screenshot at each Expected step.

> **For Codex computer use**
>
> Each case below is self-contained: preconditions, numbered steps using visible labels, and an observable expected result. Treat any mismatch in label text, numbers' presence or state as a failure and capture a screenshot.



`LIVE CAPTURE` Screen 21 — UAT starting point: the lock screen, where the tester picks REAL or PAPER and enters the demo PIN

25.2 UAT catalogue

ID	Title	Steps	Expected result
UAT-01	Unlock real book	Open app → REAL PORTFOLIO → type PIN → UNLOCK	Terminal loads; top bar shows REAL active; summary cards show S$ values
UAT-02	Wrong PIN	Type 0000 → UNLOCK	"Wrong PIN" shown; terminal stays locked
UAT-03	Operator card	Profile → set name, base SGD → Start empty → SAVE & CONTINUE	Dialog closes; greeting/profile uses name
UAT-04	Add via command	CMD: `ADD AAPL 10 195.50` → GO	AAPL row appears/updates; toast confirms; UNDO available
UAT-05	Undo	CMD: `UNDO` → GO	Previous change reverted; positions match prior state
UAT-06	Trade sheet	Positions → NVDA row → Buy → Shares 10 → save	Sheet shows You hold / Position after / New average before save; row updates after
UAT-07	Stock vs FX split	Read Unrealized, Stock P&L, FX P&L cards	Unrealized equals Stock + FX (within rounding)
UAT-08	Chart	Click VOO row → 1Y → Indicators → add RSI	Security Detail shows VOO; RSI pane appears
UAT-09	FX what-if	FX Risk → drag what-if slider to −5%	Impact on book shown negative; new value updates
UAT-10	Alert	CMD: `ALERT AAPL > 1` → GO	Alert listed; fires toast/notification (price above 1)
UAT-11	Discover tabs	Discover → US market → IPOs → Your symbols	Each tab renders with as-of; tab choice persists after reload
UAT-12	Layout	⚙ Layout → hide World Bank → DONE; drag Watchlist by ⠿	Panel hidden; order persists after reload
UAT-13	Light mode & UNPEEK	Click ◐ → click ◉ UNPEEK	Light theme applied; money figures blanked; click again restores
UAT-14	Open Kestrel	Press ⌘/	Panel opens with briefing tiles and suggestions; agent online
UAT-15	Kestrel answer	Ask "What is my currency and sector exposure?"	Tool log shows exposure tools; answer cites asOf; cost shown
UAT-16	Skill menu	Type `/` in composer	Commands and skills listed; `/pre-trade-check` selectable
UAT-17	Stop a turn	Ask a long question → press Esc	Turn stops; partial answer marked stopped
UAT-18	Delete chat	Type `/delete` → confirm	Chat removed from the Chats rail

ID	Title	Steps	Expected result
UAT-19	Council	⚖ Council → lens Balanced → "Should I trim NVDA before earnings?"	Stages Assess/Challenge/Synthesize progress; 3 cards + chair; cost shown
UAT-20	Council view switch	Press ⌘. during/after council	Toggles Cards ↔ Chain of thought
UAT-21	Switch to paper	Top bar → PAPER	Amber stripe/PAPER badge; connect panel if no keys; REAL data unchanged
UAT-22	Reject live keys	Enter key id not starting PK → Connect paper	Refused with message; nothing saved
UAT-23	Connect paper (fake)	Enter test PK key + secret → Connect paper	Desk appears; account strip shows equity; "Order stream live"
UAT-24	Market order	Ticket: BUY · SPY · Market · 1 share · Day → Buy SPY	Order fills; Positions (1); chart B marker; ledger entry
UAT-25	Limit + cancel	BUY · AAPL · Limit 1.00 · GTC → Buy; Open orders → Cancel	Dashed limit line appears then disappears; History shows cancelled
UAT-26	Kestrel ticket (ask)	Settings: Ask me first → ask Kestrel to buy 1 AAPL in paper	Pending ticket bar with Approve/Reject; Approve places order
UAT-27	Kestrel limit refusal	Settings: Max order US$ 10 → Save → ask Kestrel to buy 5 SPY	Order refused with reason; nothing sent
UAT-28	Manual edit refused in PAPER	In PAPER, CMD: `ADD AAPL 1 100`	Refused with explanation; orders are the only way in
UAT-29	Mobile layout	Viewport 390 px; scroll terminal and paper desk	No horizontal page scroll; ticket stacks below chart
UAT-30	Lock	Click 🔒	Returns to Opening Bell; API requests require PIN again

PART E

Delivery, infrastructure and operations

Domains 14–18: SDLC, infrastructure, CI/CD, observability, operations

26. SDLC and governance

`Domain 14 SDLC and governance`

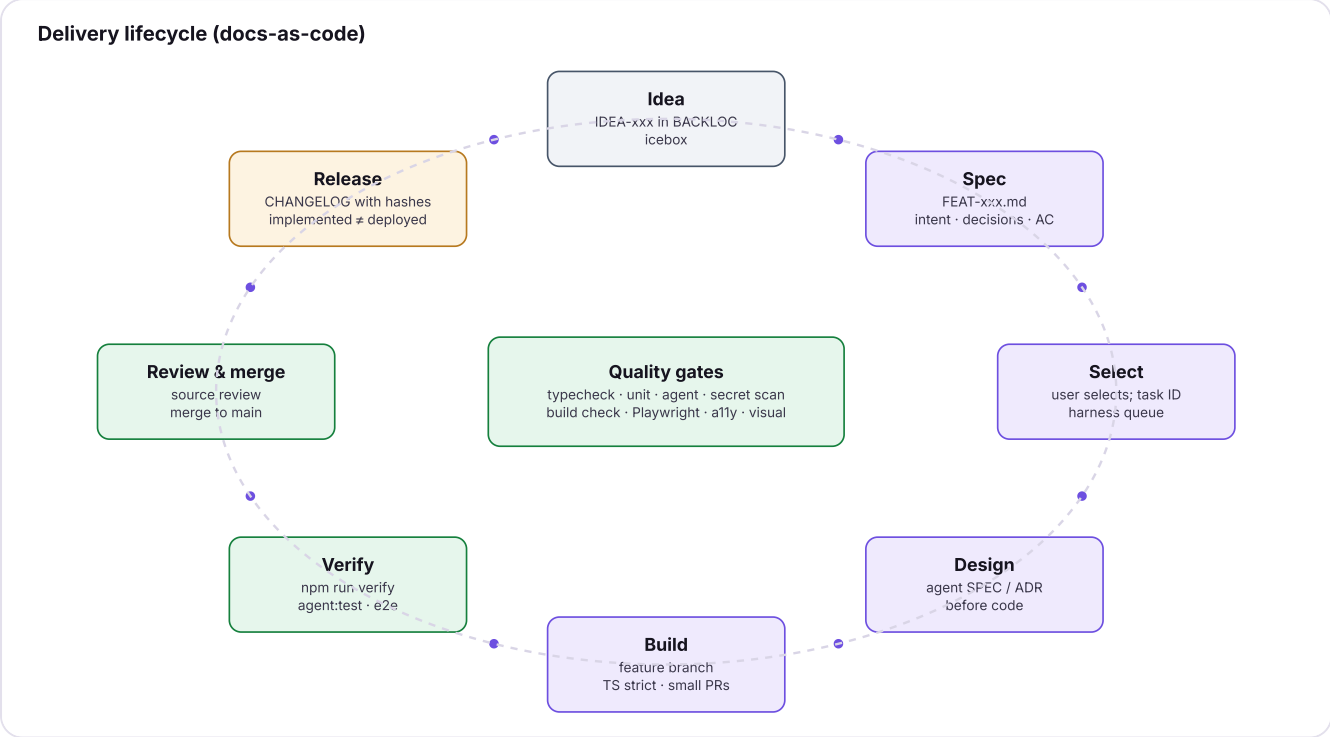


Figure 16 — Delivery lifecycle

26.1 Docs-as-code process

1. Ideas go to the **icebox** in `docs/BACKLOG.md` (IDEA-xxx); nothing is built from an idea.
2. Selected work gets a feature spec `docs/features/FEAT-xxx-<slug>.md` with intent, dated decisions, scope, out of scope and acceptance criteria.
3. Execution is scheduled in the investment-workspace harness (`harness/queue.json`, task IDs such as `PAPER-001`); user instructions override queue order.
4. Agent changes start from `agent/docs/SPEC.md`; contradictions need the user's approval; significant choices get an ADR.
5. Work happens on a feature branch, is verified, reviewed and merged to `main`.
6. `CHANGELOG.md` records integrated changes with commit/evidence hashes, distinguishing implemented, verified, merged and deployed.

26.2 Engineering standards

- TypeScript strict; erasable syntax only so Node runs `.ts` natively.
- Single-responsibility modules; `server.ts` routes only; pure math in its own modules (`returns.ts`, `portfolio.ts`).
- No LLM arithmetic anywhere in the product; deterministic tools own numbers.
- No personal data, keys or records in the repo; secret scan enforced.
- Accessible names match visible text; reduced-motion and keyboard paths for new UI.
- Commits are authored as the owner; small, reviewable changes; visual baselines refreshed deliberately.

26.3 Quality gates

Command	What it checks
`npm run typecheck`	Web, test and server TypeScript
`npm test`	Offline unit tests (server + web) with `node:test`
`npm run agent:test`	Agent typecheck, tests and secret scan
`npm run check:web`	Committed bundle matches the source build
`npm run e2e`	Playwright browser suite (fake market, fake Alpaca, fake agent)
`npm run verify`	All of the above — required before release

26.4 Engineering gates

Coding agents write features quickly, but speed does not prove a change is correct, secure or operable. Every change passes five gates, each tied to a concrete FinSight check.

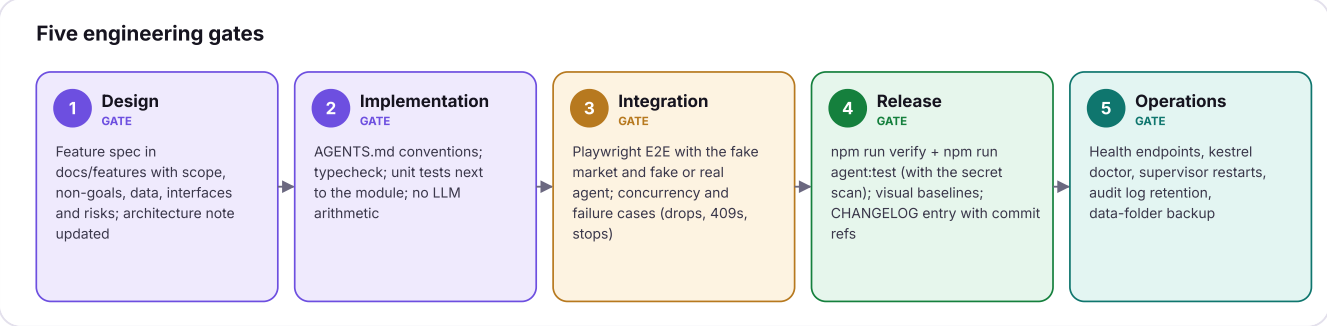


Figure 17 — The five engineering gates and the FinSight checks behind each

26.5 Branching, commits and reviews

Work happens on a feature branch (`feat/…`, `fix/…`, `mig/<card>` for frontend migration cards) in its own worktree, and merges into `main` with `--no-ff` after the gates pass. Commits are small, use conventional prefixes (`feat`, `fix`, `docs`, `mig`) and are authored as the owner. Reviews are done by the owner with AI review tools; visual baselines are refreshed only deliberately.

26.6 Technical debt management

Debt is made visible by scripts: `scripts/coupling-inventory.mjs` lists legacy couplings that the frontend migration must remove, `scripts/css-dead.mjs` finds unused CSS, and parity tests (`test/publicmath.ts`, legacy maths fixtures) keep refactored maths identical to the original.

27. Infrastructure and environments

`Domain 15 Infrastructure and cloud`

FinSight runs entirely on the owner's computer. There is no cloud infrastructure.

27.1 Topology

Element	Choice
Hosting	The owner's Mac (macOS primary; the terminal also runs on Windows and Linux)
Network exposure	Bound to `127.0.0.1`; `HOST=0.0.0.0` is an explicit opt-in for LAN access, and only then is the Host check relaxed
Inbound ports	None exposed beyond loopback
Outbound	Market data APIs, model providers, search providers, Alpaca paper
Process management	launchd LaunchAgent (`install-autostart.sh` / `uninstall-autostart.sh`); the server supervises the agent
TLS	Not needed on loopback; all outbound calls use HTTPS
Runtime check	Launcher and autostart refuse Node versions below 24

27.2 Run modes

Mode	Command	Port(s)	Data
Personal	`node server.ts` / autostart (`install-autostart.sh`)	8000 · agent 8100	`FINSIGHT_DATA_DIR` (e.g. ~/FinSightData)
Standalone (Mac)	double-click `start.command`	8002 · agent 8102	~/FinSightStandalone
Demo	`npm run demo` / `start-demo.command`	8003 · agent 8103	~/FinSightDemo (PIN 2468)
Test / UAT	`FINSIGHT_FAKE_MARKET=1 … node server.ts`	any	temporary folder

Each environment has its own port and data folder, so demos and tests can never touch real holdings. The test environment uses the fake market and fake broker, so it needs no network.

27.3 Not applicable

Containers, serverless, VPCs, gateways, object storage, infrastructure-as-code and data-residency choices do not apply: nothing is hosted. If FinSight were ever offered to others, this chapter would be the starting point for that design.

28. CI/CD and release engineering

`Domain 16 CI/CD and release engineering`

FinSight is released by merging to `main` and restarting the local server. The pipeline runs on the developer's machine.

28.1 Build

- `npm run build` bundles the React app with esbuild into `public/app/` (content-hashed chunks plus a manifest). The output is committed, so users run FinSight without installing build tools.
- `npm run check:web` rebuilds and fails if the committed bundle differs from the source. This is a bundle-consistency check on the same machine, not an independent reproducible-build verification.
- Lockfiles pin every dependency; the agent installs with `npm ci`.

28.2 Pipeline

Stage	Command	Blocks release on
Typecheck	`npm run typecheck`	Any type error in server, web or tests
Unit	`npm test`	Failing unit test
Agent	`npm run agent:test`	Agent type error, failing test, or a secret found
Bundle	`npm run check:web`	Committed bundle out of date
E2E	`npm run e2e`	Failing browser test or visual diff
All	`npm run verify`	Any of the above

28.3 Release, rollout and rollback

- **Release:** merge the verified branch into `main`, record it in `CHANGELOG.md` with commit references, restart the server (launchd).
- **Progressive enablement:** features ship behind switches — `KESTREL_COUNCIL` for the council, approval mode and limits for paper trading, `KESTREL_UI` for Intelligent UI views.
- **Service worker updates:** the shell cache name is bumped when the file set changes so open tabs pick up the new build; the chunk guard reloads safely.
- **Database migrations:** backed up before applying; a migration refuses to run while another process holds the database.
- **Rollback:** `git revert` the merge and restart; restore `agent.db.bak-v<N>` if a migration must be undone.
- **Smoke test after start:** `/api/health`, `/api/agent-status` and `kestrel doctor`.

28.4 Planned improvement

A hosted CI workflow (for example GitHub Actions) running `npm run verify` and `npm run agent:test` on every push would make the gate independent of the developer's machine. It is the first item on the engineering backlog (§34).

29. Observability and monitoring

`Domain 17 Observability and monitoring`

Observability in FinSight answers two questions: is the system healthy, and why did Kestrel give this answer?

29.1 Health and status signals

- The top bar shows **LIVE** or a stale state for market data; panels show their as-of time.
- `/api/agent-status` exposes the supervisor state (running, crashed, restarting), restart count and last exit; the Kestrel panel turns this into a readable message.
- The paper desk shows the order-stream state ("Order stream live" or reconnecting).

29.2 Agent traces and logs

Every model call, tool call and data request is logged with timing and cost to `agent.db` and redacted JSONL logs (14-day retention). The UI exposes a per-answer trace; `/logs` shows recent errors; `/cost` reports spend by day or month.

29.3 Structured logs and correlation

The agent's audit log writes one JSON line per API request, model request and response, tool call, FinSight call, MCP call and error, in daily files. Each line carries `turnId`, `threadId` and request ids propagated with `AsyncLocalStorage`, so one answer can be traced end to end. Lines are redacted before writing and large fields are capped.

29.4 Business metrics

Cost per answer, per day and per month (`/cost`), steps, tool calls and latency per turn are recorded in the trace and shown in the UI. Council runs record per-stage status and readiness.

29.5 Not applicable at this scale

Centralised log collection, metrics dashboards, distributed tracing across hosts, real-user monitoring and paging are not needed for one local user. Price alerts are a product feature, not an operational alert.

30. Production operations and support

`Domain 18 Production operations and support`

The owner is also the operator. This chapter lists what is needed to run, fix and maintain FinSight.

30.1 Configuration

Variable	Purpose
`PORT`, `HOST`	Listen port / host (loopback)
`FINSIGHT_DATA_DIR`	Data folder outside the repo
`AGENT_URL`, `AGENT_AUTOSTART`, `AGENT_DIR`, `AGENT_SHELL_ENV`	Agent location and supervision
`FINSIGHT_FAKE_MARKET`	Offline deterministic market + fake Alpaca for tests/demo
`FINSIGHT_CACHE_PERSIST`	Persist market cache to disk
`LOCK_IDLE_MIN`	Idle auto-lock minutes
`NOTIFY`, `ALERT_INTERVAL`	Desktop notifications and alert polling
`ALPACA_PAPER_KEY_ID`, `ALPACA_PAPER_SECRET_KEY`	Paper keys via environment (alternative to UI)
`DEEPSEEK_API_KEY`, `ANTHROPIC_API_KEY`, `TAVILY_API_KEY`	Model and search keys (prefer Keychain)
`KESTREL_UI`	Intelligent UI views setting

30.2 Runbook

Task	How
Install agent	`npm run agent:install`
Health	`node agent/src/cli.ts doctor`; `/api/agent-status`
Logs	`agent.log`, `agent/logs/*.jsonl`; `/logs` in Kestrel
Reset PIN	Delete `auth.json` in the data folder (holdings kept)
Reset demo	`npm run demo:reset`
Port clash	`PORT=9000 node server.ts`
Backup	Copy the data folder; `portfolio.json` + `undo.json` + `agent/agent.db`
Register MCP	`claude mcp add finsight -- node <finsight>/agent/src/cli.ts mcp`

30.3 Incident handling

Symptom	First check	Fix
Kestrel panel says offline	`/api/agent-status`, `agent.log`	`npm run agent:install`; check keys with `kestrel doctor`
Panels stale	Top bar status, panel as-of time	Usually a source outage; data refreshes when it returns
Paper stream not live	Desk account strip	Reconnect, or Disconnect and Connect paper again
Port in use	Error at start	Start on another port (`PORT=9000 node server.ts`)
Forgotten PIN	—	Delete `auth.json` in the data folder
Bad data after a change	Undo, `.corrupt.json`, backups	Undo (25 steps) or restore the data folder

30.4 Patching and upgrades

Dependencies are upgraded deliberately, one change at a time, with the full gate. Node 24 or newer is enforced by the launcher. After an upgrade the service worker picks up the new shell automatically.

30.5 Support

The interactive guide (`docs/guide/finsight-guide.html`) includes a troubleshooting scene; `/help` and `/logs` in Kestrel show capabilities and recent errors.

PART F

Business, governance and sustainability

Domains 19–23: cost, privacy and compliance, documentation, analytics, maintainability

31. Cost engineering and FinOps

`Domain 19 Cost engineering and FinOps`

FinSight's infrastructure cost is zero: it runs locally and uses free, keyless data. The only variable cost is AI model usage, which is engineered to be small, visible and capped.

31.1 Cost model

Cost	Amount	Control
Hosting, compute, storage	US$0 (owner's machine)	—
Market and macro data	US$0 (public sources)	Cache to stay within rate limits
Paper broker	US$0 (Alpaca paper)	—
Model tokens	≈ US$0.0003–0.0008 per simple DeepSeek turn	Per-turn and per-day caps
Web search	Provider pricing (Tavily, Brave)	Searches per turn limit

31.2 Budgets and caps

Limit	Default	Editable
US$ per reply	0.25	Yes (`/limits` or the Today spend)
US$ per day	5	Yes
Council research calls	8	Yes; optional council turn ceiling
Steps / seconds per reply	25 / 300	Yes
Searches / pages per reply	20 / 25	Yes

Limits are changed only by the owner; Kestrel's token cannot change them.

31.3 Cost control mechanics

- **Reservation before every billable call**, reconciled after, with a persisted daily ledger, so parallel calls (for example council specialists) cannot overshoot a cap.

- **Prompt caching** lowers repeat input cost (FEAT-005).
- **Model choice by cost tier**: cheap default, stronger models on demand.
- **Attribution:** cost is recorded per answer and per thread and summarised by day and month (`/cost`).

32. Privacy, legal and compliance

`Domain 20 Privacy, legal and compliance · Domain 22 Product analytics and experiments`

Privacy is a product goal (G3). FinSight keeps financial data on the owner's machine and sends out only what each feature needs.

32.1 What data goes where

Data	Stays local	Leaves the machine
Holdings, trades, cash, records	Yes, in the 0600 data folder	Only as tool results inside a Kestrel question, to the chosen model provider
Symbols	—	Sent to public market APIs to fetch quotes
Paper orders	Mirror book local	Sent to Alpaca paper
Web research queries	—	Sent to the chosen search provider
Keys and PIN	Yes (Keychain, 0600 files, scrypt hash)	Never

32.2 Principles applied

- **Data minimisation:** no account, no telemetry, no cloud sync.
- **Purpose limitation:** data is used only to answer the owner's questions and run the owner's book.
- **Retention and deletion:** chats can be deleted; logs expire after 14 days by default; the owner can delete the data folder at any time.
- **Repository hygiene:** `AGENTS.md` forbids personal data, keys and records in the repo; broker exports (`*.csv`) are git-ignored; demo and test data are synthetic.

32.3 Regulation and terms

- Singapore's PDPA governs organisations; an individual managing their own data in a personal capacity is outside its scope. If FinSight were offered to other users, consent, access and correction workflows and cross-border transfer terms would apply.
- Data-source terms: Forex Factory's feed is for personal use and is not redistributed; Yahoo endpoints are unofficial and used for personal analysis only.
- FinSight is MIT-licensed; dependency licences are permissive.
- Not investment advice: FinSight and the council state this in the product.

32.4 Product analytics and experimentation

Domain 22 was evaluated and is **intentionally not implemented**: product analytics would mean telemetry, which conflicts with G3. Quality is measured offline instead — council evaluation metrics (`council-`

`metrics.ts`), per-turn cost and latency traces, and UAT results. If FinSight ever had other users, analytics would be opt-in and privacy-preserving.

33. Documentation and knowledge management

`Domain 21 Documentation and knowledge`

Documentation lives with the code and changes in the same commit as the behaviour it describes.

33.1 Document set

Concern	Where it lives
Product requirements	`docs/product/PRD.md` ; §5–§9 of this document
Architecture	`docs/architecture/SYSTEM.md` , `finsight-architecture.md` , `kestrel-architecture.md` ; §10
Decision records	`agent/docs/ADR-THINKING-EFFORT.md` ; decisions inside feature specs; §10
Agent specification	`agent/docs/SPEC.md` , `COUNCIL.md` , `INTELLIGENT-UI.md` , `UI-QUALIFICATION.md`
Data model	§14; `agent/docs/SPEC.md`
API	§15; `DOCS.md`
Security	§20; `agent/docs/SPEC.md` §10
Test plan	§24, §25; `web/e2e/COVERAGE.md`
Deployment and operations	§27–§30; `README.md` , `START-HERE.md`
Roadmap	`docs/BACKLOG.md` ; `docs/features/FEAT-001…013` ; §34
Rules for coding agents	`AGENTS.md`
User guide and demo	`docs/guide/finsight-guide.html`

33.2 Onboarding

`START-HERE.md` and `README.md` take a new user from download to a running terminal; `npm run demo` gives a safe sandbox. A new contributor or coding agent reads `AGENTS.md` , this document, then the feature spec for the work at hand.

33.3 Ownership and versioning

The owner maintains all documents. This master document is generated from `docs/master/src` (content, diagrams and screenshots) so the PDF and Word versions never drift, and it states the code baseline it reflects.

33.4 Repository structure

```
finsight/
├── server.ts            Node HTTP server: routes, headers, session gate, proxies
├── lib/                 Domain modules (portfolio, market, macro, paper, auth, cache…)
├── web/                 React 19 + TS source
│   ├── src/features/    one folder per panel (positions, kestrel, paper, discover…)
│   ├── src/shell/       boot, layout, chart hooks
│   └── e2e/             Playwright specs, fake agent, fixtures
├── public/              served as-is: index.html, app/ bundle, chart.js, styles, sw.js
├── agent/               ↗ Kestrel (TypeScript)
│   ├── src/             api, chat, brains, tools, store, sandbox, mcp, obs
│   ├── skills/          bundled playbooks
│   ├── plugins/brains/  one manifest per model
│   └── docs/            SPEC, COUNCIL, INTELLIGENT-UI, ADRs
├── shared/              contracts shared by server/agent/web (council lenses)
├── scripts/             build-web, demo, checks
├── test/                node:test unit suites and fixtures
├── docs/                BACKLOG, features (FEAT-xxx), architecture, product PRD, guide,
master
└── AGENTS.md            rules for coding agents
```

34. Maintainability and future evolution

`Domain 23 Maintainability and evolution`

FinSight is built to be changed safely by one owner working with coding agents.

34.1 Modularity and separation of concerns

- Pure maths in `lib/` modules with unit and parity tests; `server.ts` only routes.
- UI panels as independent islands; shared primitives in `web/src/ui`.
- One agent tool registry used by Kestrel, the council and MCP.
- Contracts shared between server, agent and web in `shared/`.

34.2 Extensibility

Brains, skills, strategies and search providers are plugins with manifests. Adding a model or a playbook does not change the core; the registry is reloaded as an immutable snapshot.

34.3 Ongoing evolution

- **Frontend migration:** legacy panels are being moved to React islands card by card, with a coupling inventory, parity tests and visual baselines (`docs/frontend-migration`).
- **Backward compatibility:** data files are upgraded on load; the agent API is versioned; council contracts are versioned.
- **Data portability:** CSV export of holdings, ledger and journal; data files are plain JSON and SQLite.
- **Handover:** this document, `AGENTS.md` and the guide reduce the risk of single-owner knowledge.

34.4 Feature status

Spec	Feature	Status (2026-11-01)
CORE-001	Kestrel research agent core	Integrated 2026-10-24
IUI-001 / START-001	Intelligent UI views; briefing start screen	Integrated 2026-10-26
FEAT-002	Investment lenses & council mode	Merged, verified, personally activated; quality work open (COUNCILQUALITY-001)
FEAT-003	Chat deletion, /delete, /clear, bulk clear	Merged — verified
FEAT-005	Prompt-cache optimisation	Merged — verified
FEAT-006 / 007 / 008	New-symbol trades; account transfers; IBKR fill import	Merged — verified
FEAT-009	Kestrel terminal redesign (panel states, light mode, council views)	Merged (PR #1)

Spec	Feature	Status (2026-11-01)
FEAT-010	Market Discover: movers, US movers, IPO calendar	Merged locally; not deployed
FEAT-012	Paper trading, trade desk, Kestrel paper orders and limits	Merged to main (`aa373b7`); live run with real paper keys pending
FEAT-013	Agentic trading cycle	Plan, not selected
FEAT-001 / 011	IBKR MCP live sync; agent data sources	Deferred
IDEA-001…004	PDF memo export, ETF look-through, multi-broker, voice input	Icebox

34.5 Engineering backlog

Priority	Item	Domain
P1	Hosted CI running `npm run verify` and `npm run agent:test` on every push	CI/CD
P1	Strict Content-Security-Policy header	Security
P2	Dependency audit in the release gate	Security
P2	Documented and rehearsed data-folder backup and restore	Data, reliability
P3	`docs/adr/` with the key past decisions	Architecture, documentation
P3	OpenAPI description of `/api` and `/v1`	APIs, documentation

35. Risks, assumptions and constraints

Type	Item	Mitigation / note
Risk	Unofficial Yahoo endpoint changes or rate-limits	Cache, backoff, stale flags; sources isolated in `market.ts`
Risk	Model hallucination or overconfidence	Tool-only numbers, citations, council gaps, figure flags, user decides
Risk	Prompt injection via web/docs	Untrusted wrapping; no write tools on the real book
Risk	Agent over-trading paper	Ask mode default, limits, PIN-only settings
Risk	Provider cost spikes	Turn and daily caps with reservation
Assumption	Single trusted user on a personal machine	Controls target other processes/pages, not other users
Assumption	FinSight running when the agent needs facts	Agent degrades with FINSIGHT_DOWN
Constraint	Forex Factory feed personal-use only; no redistribution	Local use only; disclaimer in README
Constraint	US-only market movers and IPOs	SG/HK deferred
Constraint	No ETF look-through yet	Exposure answers must say so

Disclaimer

FinSight is a personal tool and not investment advice. It is not affiliated with Bloomberg, Yahoo, Interactive Brokers, Alpaca, Forex Factory or the Federal Reserve. Data is provided as-is.

FINSIGHT

Appendices

Appendix A — Glossary

Term	Meaning
Base currency	Currency all totals are reported in (default SGD)
Stock P&L / FX P&L	Part of unrealized gain from the price move / from the exchange-rate move
Blended entry	Weighted-average exchange rate paid for foreign holdings
TWR / XIRR	Time-weighted return (ignores cash-flow timing) / money-weighted personal return
VaR	Value at risk; here 1-week, 1σ and 95% from recent FX volatility
Lot	One purchase with quantity, price, FX and time
Brain	Model plugin used by Kestrel for a turn
Tool	Function the model can request; executed by deterministic code
Skill	Playbook in SKILL.md format invoked with `/name`
Context pack	Dated snapshot of the user's situation built by code for each turn
Lens	Analytical emphasis for the council (Balanced, Growth, Speculative, Hedge Fund, Macro)
Ticket	A Kestrel paper order waiting for Approve/Reject
UNPEEK	Privacy mode that blanks money figures
MCP	Model Context Protocol; exposes tools to external AI clients

Appendix B — Terminal commands

Command	Effect
`ADD SYM QTY PRICE [DATE]`	Buy/add (averages in); optional date fetches historical FX
`SELL SYM QTY [PRICE]`	Sell; realized P&L recorded
`DEL SYM`	Stop tracking
`CASH CCY AMOUNT`	Set cash
`UNDO`	Undo last change
`LEDGER`	Open trades ledger
`Q SYM` / `SYM`	Load symbol detail
`WATCH` / `UNWATCH SYM`	Watchlist
`ALERT SYM > | < PRICE`	Price alert
`NEWS SYM`	News
`WHT MKT PCT`	Withholding rate
`MKT` · `HELP`	Refresh markets · help

Appendix C — Kestrel commands

Command	Effect
`/new` · `/clear` · `/delete`	New chat · fresh chat keeping history · delete (asks first)
`/model <id>`	Switch brain (e.g. `deepseek-pro`)
`/cost [month]`	Spend
`/limits`	Spending and tool limits
`/strategies` · `/skills` · `/logs` · `/help`	Library · installed skills · recent errors · capabilities
`/committee <question>`	Run the council once
`/pre-trade-check` · `/position-review` · `/earnings-review` · `/why-did-it-move` · `/strategy-scan`	Skills
⌘/ · ⌘⇧/ · ⌘. · Esc	Toggle panel · full page · council view · stop

Appendix D — Traceability matrix

Story	Functional requirements	Automated evidence	UAT
US-A1/A2	FR-ACC-1…4	lock, session, PAPER specs	UAT-01, 02, 21, 30
US-A3/A4	FR-ACC-5	startup, demo safety	UAT-03
US-B1/B2	FR-PF-1, FR-PF-2	portfolio unit tests, summary visual	UAT-07
US-B3/B4/B5	FR-PF-3…7	add-sell, import-export, trades, ledger, OPENFEATURES	UAT-04, 05, 06
US-B6/B7/B8	FR-AN-1…3	returns/performance units, panels spec	UAT-09
US-C1…C5	FR-MD-1/2, FR-DS-1, FR-MC-1, FR-AL-1	chart, DISCOVER, F-specs, visual	UAT-08, 10, 11, 12, 13
US-D1…D8	FR-K-1…11	G.agent, KESTRELUX, CHATDELETE, LIMITS, IUI, agent tests	UAT-14…18
US-E1…E4	FR-C-1…6	COUNCIL.agent, COUNCIL.motion, council real-backend	UAT-19, 20
US-F1…F4	FR-P-1…7	PAPER.spec, paper unit tests vs fake broker	UAT-21…29

Appendix E — References

- `README.md`, `DOCS.md`, `AGENTS.md` — overview, developer/API reference, agent rules
- `docs/architecture/SYSTEM.md`, `finsight-architecture.md`, `kestrel-architecture.md`
- `agent/docs/SPEC.md` (SRS), `COUNCIL.md`, `INTELLIGENT-UI.md`, `ADR-THINKING-EFFORT.md`
- `docs/product/PRD.md`, `docs/BACKLOG.md`, `docs/features/FEAT-001…013`
- `docs/guide/finsight-guide.html` — interactive demo guide and user manual
- `web/e2e/COVERAGE.md` — browser test coverage map